Bachelorprojekt

SafeRoute



AU-ID	Navn	Studieretning
AU737123	Kevin Pham	SW
AU755585	Bleron Gashi	SW

Indholdsfortegnelse

Abstract 3
1. Introduktion 4
1.1 Baggrund 4
1.2 Eksisterende arbejde 4
1.3 Systemskitse 5
1.4 Problemformulering / Hypotese 5
2. Udviklingsproces og metode 6
3. Brugerkrav 7
3.1 Systemkontekst og aktører 7
3.2 Funktionelle krav 8
3.2.1 Elicitering og analyse 8
3.2.2 Specifikation (MoSCoW) 8
3.3 Ikke-funktionelle krav 9
3.3.1 Elicitering og analyse 9
3.3.2 Specifikation 10
3.4 Systemets domænemodel 11
4. Systemarkitektur 12
4.1 Systemets ydre grænser og relationer 12
4.2 Analyse af arkitektur 12
4.2.1 Ruteberegning under belastning (NF1, NF6) 13
4.2.2 Geofence-baseret notifikation vs. serverstyret push (NF3, NF11) 13
4.2.3 Frisk trygsedsscore fra bekræftede indrapporteringer (NF4, NF5) 14
4.2.4 Placering af differential privacy-laget (NF11, NF13) 14
4.3 Systemets struktur og grænseflader 15
4.3.1 Interaktion mellem delsystemer 16
4.3.2 Interfaces og protokoller 17
4.3.3 Datamodeller og persistering 17
4.3.4 Valg af teknologistack 18
Backend API: FastAPI (Python) 18
Mobile Client: React Native med Expo 18
Datalag: PostgreSQL med PostGIS 19
Rutemotor: OSRM (selvhostet) 19
Async Worker: enkelt baggrundsproces frem for Celery/Redis ved lancering 19
Kildeliste 20
Vejleder Noter 21

Abstract

1. Introduktion

1.1 Baggrund

Hver aften åbner millioner af mennesker en kortapp som Google Maps og får præcis ét svar: den hurtigste vej hjem til fods. Appen tager ikke stilling til, om ruten føles tryg, selvom trygheden for mange spiller lige så stor en rolle som rejsetiden.

Forestil dig en person, der går ud fra en togstation kl. 21.47. Appen siger, at den hurtigste vej hjem er elleve minutter, lige igennem et kvarter uden gadelys og uden andre mennesker, og der findes ikke et andet forslag på skærmen. Personen går ruten alligevel, for alternativet er ikke at vide, hvor man ellers skulle gå hen. Det samme gælder kvinder på vej hjem fra arbejde, turister i en fremmed by, ældre der løber et ærinde efter mørkets frembrud, og forældre der følger et barn hjem: det eneste værktøj, der guider dem gennem byen, har aldrig spurgt, om vejen føles tryg.

Denne utryghed er ikke sjælden, den er normen, både internationalt og i en dansk kontekst. Gallups Global Law and Order-undersøgelse (120+ lande) viser, at 33% af kvinder ikke føler sig trygge ved at gå alene om natten i deres egen by; i Storbritannien fandt Office for National Statistics tilsvarende 49%. Danske tal peger samme vej: Justitsministeriets Tryghedsundersøgelse 2023 finder en signifikant kønsforskel i oplevet utryghed, og Det Kriminalpræventive Råd har opgjort, at cirka hver ottende kvinde, mod hver 20. mand, er bekymret for at gå i sit nærområde efter mørkets frembrud. En DR-artikel fra samme periode beskriver, hvordan det viser sig i konkret adfærd, f.eks. ved at gå med nøgler mellem fingrene eller ringe op med en falsk samtale for at virke mindre sårbar.

Problemet er ikke unikt for storbyer. En Stiften-artikel (2022) beskriver to unge kvinder fra Aarhus, hvor kun den ene tør gå alene hjem om natten. TrygFondens forskningschef peger på, at utrygheden stiger blandt unge (18-24 år) og falder blandt ældre, og Københavns Kommunes Tryghedsundersøgelse 2025 viser tilsvarende, at trygheden varierer markant mellem bydele og aldersgrupper, selvom den samlet set er høj og stigende.

Utrygheden er ikke kun en følelse uden konsekvens. Fodgængere fravælger aktivt ruter, de opfatter som utrygge, uanset rejsetid. Et review af fodgængeres rutevalg (Monash/Cities, 2023) finder, at kvinder systematisk undgår dårligt oplyste ruter om aftenen, uafhængigt af den faktiske kriminalitetsrisiko på stedet. Det er den oplevede tryghed, ikke tallene, der styrer adfærden. Konsekvensen er, at folk undgår at gå, betaler for ture de ellers ikke ville tage, eller vælger ruter de ved er dårligere, blot fordi de er velkendte, og det sker i det skjulte, uden at nogen af de værktøjer, folk er afhængige af, registrerer det.

1.2 Eksisterende arbejde

De mest udbredte navigationsapps, Google Maps og Apple Maps, optimerer udelukkende for rejsetid og afstand. Et studieprojekt om natlig sikkerhed (Coolight, 2025) bemærker, at disse apps "prioriterer effektivitet over sikkerhed" og i vid udstrækning er designet med bilister for øje, ikke fodgængere. I mangel af et bedre redskab improviserer folk selv. De ringer til en ven og bliver på telefonen hele vejen hjem, vælger en længere men velkendt rute frem for den ukendte og hurtige, eller vurderer ud fra mavefornemmelse, hvor godt oplyst og befærdet en gade ser ud i mørket.

Der findes dog en række nichetilbud internationalt, der forsøger at adressere netop dette:

- ZoneScout tilbyder et kriminalitetskort og ruter, der undgår "high-risk"-områder, ud fra 30 dages officielle politidata.
- SafePath har lignende funktionalitet samt en nødknap, der kan underrette politiet med brugerens placering.
- Safest Way (Storbritannien, udviklet som en del af et ph.d.-projekt ved UCL) går videre og bruger en AI-model til at kombinere belysningsdata, CCTV-dækning og billedanalyse af gadebilleder for at vurdere den oplevede tryghed på en gade, ikke kun den registrerede kriminalitet.

Et nyere studie peger desuden på, at digitale sikkerhedsværktøjers effekt på kønsforskelle i oplevet tryghed fortsat er underbelyst. Belysnings- og nyhedsdata findes i øvrigt allerede, spredt over kommunale open-data-portaler, OpenStreetMap-tags og lokale nyhedsmedier, men ingen af ovenstående apps samler dem til noget brugbart, før man overhovedet tager det første skridt.

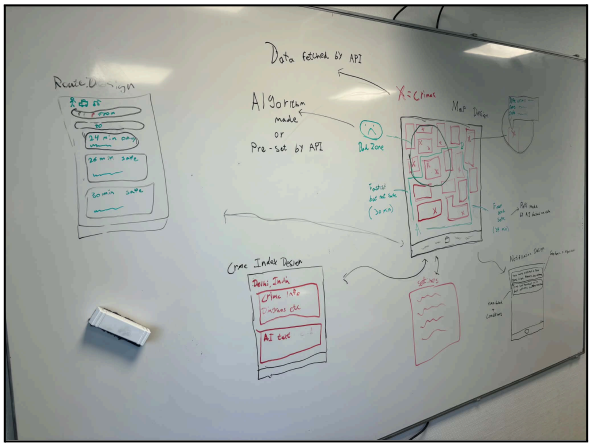
Fælles for de eksisterende løsninger er, at de henter deres data fra officielle, institutionelle kilder (politidatabaser, kommunale registre) frem for fra brugerne selv i realtid, at de ikke forklarer brugeren, hvorfor en given rute vurderes utryg, og at ingen af dem findes for danske byer — Safest Way dækker eksempelvis kun London, Nordirland, Birmingham og Manchester. Det er den mangel, SafeRoute adresserer.

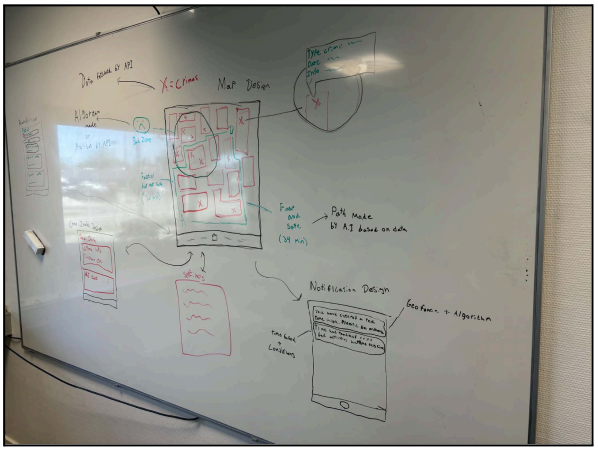
1.3 Systemskitse

Ideen er et navigationssystem til fodgængere, der ud over den hurtigste rute også kan foreslå den tryggeste, og lade brugeren selv vælge, hvad der betyder mest lige nu. Systemet henter information fra flere niveauer: et overordnet trygheds-/kriminalitetsbillede for byen eller bydelen som helhed, baseret på offentligt tilgængelige data, samt en mere detaljeret vurdering af den enkelte gade, der primært bygger på belysningsdata og indrapporteringer fra andre brugere i realtid. Markerer flere brugere en gade som utryg, justerer systemet automatisk sine anbefalinger.

Brugeren møder systemet som en almindelig navigationsapp med et kort, hvor to ruter kan sammenlignes side om side, f.eks. "30 min, utryg" over for "34 min, meget tryg", men kan derudover spørge appen "hvorfor" en rute anbefales, f.eks. "denne gade er dårligt oplyst, og tre brugere har for nylig rapporteret ubehag her". Mens brugeren går, følger appen løbende med og advarer, hvis vedkommende bevæger sig ind i et utrygt område, eller hvis klokken passerer et tidspunkt, hvor utryghed historisk set stiger. For turister og tilflyttere tænkes desuden en søgbar oversigt på by-niveau, der opsummerer, hvor trygt et område generelt opleves.

Systemet er tænkt som en ekstra dimension oven på den navigation, folk allerede kender, ikke en ny app man skal lære fra bunden: tryghed vises synligt og forklaret ved siden af rejsetiden, uden at brugerne skal opgive vanen, de allerede har. Fordi vurderingen opdateres løbende af brugernes egne rapporter, afspejler den, hvad der sker lige nu, og det forudsættes, at de bidragende brugere kan gøre det uden at deres identitet kan spores tilbage til dem.





1.4 Problemformulering / Hypotese

Projektet undersøger, hvordan man kan udvikle et navigationssystem til fodgængere, der – ud over rejsetid – tager hensyn til, hvor trygt en given rute opleves at være. Fodgængere i ukendte eller utrygge gader, herunder i særdeleshed kvinder, turister og ældre, har i dag ikke et samlet redskab til at vurdere, hvilken rute der reelt er tryggest. Det skyldes, at de udbredte navigationsapps ikke tager højde for oplevet tryghed i deres rutevalg, og at de få kommercielle alternativer, der findes, ikke forklarer brugeren, hvorfor en given rute eller gade vurderes utryg.

Hypotesen er, at det er muligt at kombinere offentligt tilgængelige data – på både by-/bydelsniveau (f.eks. kriminalitetsstatistik) og gadeniveau (f.eks. belysning) – med brugernes egne realtidsindrapporteringer, så systemet kan:

- foreslå en rute, der balancerer hastighed og oplevet tryghed efter brugerens eget ønske, og
- forklare brugeren, hvorfor en given rute eller gade vurderes mere eller mindre tryg.

Antagelsen er, at brugere derved sættes i stand til at træffe mere informerede og trygge valg, uden at skulle opgive den navigationsvane, de allerede har.

2. Udviklingsproces og metode

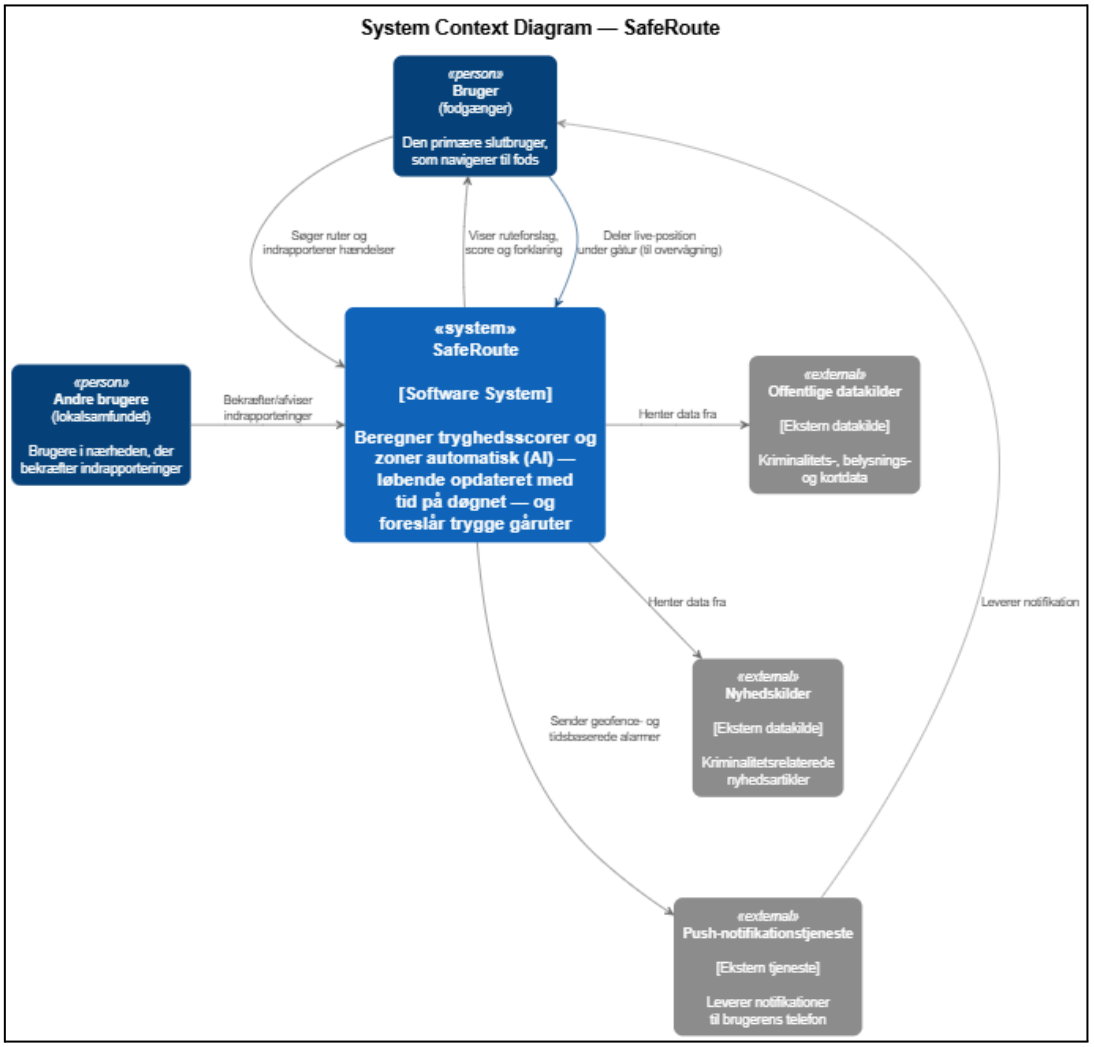
3. Brugerkrav

3.1 Systemkontekst og aktører

SafeRoute opererer i en bymæssig kontekst, hvor fodgængere til daglig skal træffe et rutevalg uden at have et samlet overblik over, hvor trygt de enkelte gader og områder er. Systemet henter derfor data fra offentlige kilder og stiller dem til rådighed for brugeren i en form, der kan bruges i planlægningen af en rute.

Følgende aktører er identificeret:

- **Bruger (fodgænger)** = den primære aktør. Søger ruter, vælger mellem hurtigst og tryggest, indrapporterer hændelser, justerer sine indstillinger og deler sin live-position med systemet, mens vedkommende går.
- **Andre brugere (lokalsamfundet)** = samme rolle som Bruger, men set fra perspektivet af de brugere, der bekræfter eller afviser andres indrapporteringer i deres nærområde.
- **Offentlige datakilder og nyhedskilder** = eksterne systemer, der løbende leverer kriminalitets-, belysnings-, kort- og nyhedsdata til SafeRoute, men som ikke selv anvender systemet.
- **Push-notifikationstjeneste** = den eksterne tjeneste (f.eks. Expo/Apple/Google's push-infrastruktur), som SafeRoute sender geofence- og tidsbaserede alarmer igennem, og som leverer dem videre til Brugerens telefon.



3.2 Funktionelle krav

3.2.1 Elicitering og analyse

De funktionelle krav udledes af fire veje. For det første tages der udgangspunkt i problemformuleringen fra kapitel 1: kravene skal samlet sikre, at det system, der skal udvikles, giver en fodgænger et reelt valg mellem en hurtig og en tryg rute, som skal være forklaret og løbende opdateres.

For det andet konkretiseres kravene gennem brugsscenarier for de målgrupper, projektet retter sig mod: kvinder der går alene om aftenen, ældre, turister i ukendte områder, samt personer med angst eller nedsat mobilitet. Det inkluderer scenariet fra baggrundsafsnittet med en bruger, der går fra togstationen sent om aftenen, samt tilsvarende scenarier for en turist i en ukendt by og en bruger, der observerer en hændelse undervejs.

For det tredje holdes kravene op imod de eksisterende løsninger, der er gennemgået i afsnit 1.2 for at sikre, at det planlagte system som minimum skal dække det, brugere allerede forventer af en rute-app, og at de identificerede svagheder ved disse løsninger (statiske data, manglende forklaring, ingen dansk dækning) adresseres som konkrete krav frem for blot som hensigtserklæringer.

For det fjerde tages der udgangspunkt i en indledende produktvision for SafeRoute, som skitserer fem centrale skærmbilleder, systemet skal bestå af: Kort, Rute, Nyheder, Bytryghedsoversigt og Indstillinger, samt den funktionalitet, hvert af dem skal understøtte. Denne vision formaliseres og prioriteres af kravene i dette kapitel. De fem skærmbilleder konkretiseres desuden gennem en række whiteboard-skitser, der skal sikre, at det samlede funktionelle omfang hænger sammen som én brugerrejse, fra en bruger åbner appen, til vedkommende er i gang med at gå en valgt rute.
Analysen giver følgende overordnede funktionelle områder, som de detaljerede krav i afsnit 3.3.2 er struktureret omkring:

- Finde og sammenligne ruter på tid og tryghed.
- Forstå, hvorfor en gade eller rute er vurderet, som den er.
- Indrapportere og verificere hændelser i realtid.
- Få et hurtigt tryghedsoverblik over en by, man ikke kender.
- Blive advaret undervejs, i takt med at trygheden ændrer sig i realtid og med tid på døgnet.

3.2.2 Specifikation (MoSCoW)

UC1: Find og vælg tryg rute

Felt	Beskrivelse
Primær aktør	Bruger (fodgænger)
Mål	Brugeren finder og vælger en rute, der afspejler den ønskede balance mellem rejsetid og tryghed.
Forudsætninger	Brugeren har åbnet SafeRoute. Enten er lokationstilladelse givet, eller Brugeren er villig til at søge start og destination manuelt.
Succeskriterium	En rute er valgt og tegnet på kortet, og Brugeren er klar til at gå.

Hovedforløb:

1. Brugeren åbner SafeRoute.
2. Brugeren angiver startpunkt, enten ved at bruge sin nuværende position eller ved søgning.
3. Brugeren angiver destination ved søgning.
4. Systemet beregner og viser en liste af ruteforslag, hver med rejsetid og en trygshedsscore fra 1-10, f.eks. "Rute A: 30 min · Tryghed: Middel (6,2/10)".

5. Brugeren sorterer forslagene efter tid eller tryghed (valgfrit).
6. Brugeren vælger ét ruteforslag.
7. Systemet tegner den valgte rute på kortet.
8. Brugeren begynder at gå.

Udvidelser:

- 4a. Der kan ikke beregnes nogen rute (f.eks. ingen sammenhængende gangforbindelse): systemet viser en fejlbesked og beder Brugeren justere start eller destination.
- 2a. Lokationstilladelse er ikke givet: systemet beder om tilladelse eller lader Brugeren søge startpunktet manuelt.

UC2: Indrapportér hændelse

Felt	Beskrivelse
Primær aktør	Bruger (fodgænger)
Mål	Brugeren advarer andre brugere om en utryg situation nær sin position.
Forudsætninger	Brugeren befinder sig i appen med en kendt nuværende position.
Succeskriterium	Hændelsen er gemt og vist som ubekræftet på kortet for andre brugere i nærheden.

Hovedforløb:

1. Brugeren trykker på rapportér-knappen.
2. Systemet henter Brugerens nuværende position.
3. Systemet kontrollerer, om Brugeren er logget ind.
4. Brugeren vælger hændelsestype (chikane, overfald, tyveri, mistænkelig adfærd eller andet) og tidspunkt (nu, 15 min, 30 min eller 1 time siden), og tilføjer eventuelt en kort beskrivelse.
5. Brugeren indsender rapporten.
6. Systemet gemmer rapporten og viser den som ubekræftet på kortet.

Udvidelser:

- 3a. Brugeren er ikke logget ind: systemet beder Brugeren logge ind eller oprette en konto, før rapporten kan indsendes. Afbryder Brugeren login, oprettes rapporten ikke.
- 2a. Lokationstilladelse er ikke givet: systemet beder om tilladelse, før rapporten kan oprettes.

Den primære aktør, Brugeren, forventes at kunne bruge SafeRoute, som man kender det fra almindelige kortapps: indtast start og destination, se ruteforslag, vælg en rute. Forskellen er, at hvert ruteforslag også bærer en tryghedsscore og en kort forklaring, så valget står mellem tid og tryghed i stedet for kun tid. Da en del af datagrundlaget kommer fra brugerne selv, er det desuden vigtigt, at systemet løbende kan modtage og verificere indrapporteringer, uden at det kan misbruges til at sprede falske alarmer.

MoSCoW-prioriteret:

#	Krav	Prioritet	Information
F1	Finde og vælge rute sorteret på tid/tryghed	Must have	Se UC1 for det fulde forløb.
F2	Indrapportere hændelse (kræver login)	Must have	Se UC2 for det fulde forløb.
F3	Verificere andres indrapporteringer	Should have	En indrapportering skal bekræftes af flere nærliggende brugere, før den vægtes fuldt i tryghedsscoren. Indtil da vægtes den lavere, for at modvirke spam og fejlagtige rapporter.
F4	Se bytryghedsoversigt for en valgfri by	Should have	Viser et kriminalitets- og tryghedsindeks for den valgte by, suppleret af detaljerede delmålinger, en letforståelig tekstuel opsummering af

			områdets generelle tryghedsbillede samt information om datagrundlagets aktualitet.
F5	Realtidsovervågning af gåtur, med geofence- og tidsbaserede push-notifikationer	Should have	Brugerens position følges løbende, mens en rute gås. Der advares både ved indtræden i et utrygt område og på tidspunkter, hvor utryghed historisk set stiger.
F6	Justere risikoprofil og notifikationsindstillinger	Should have	Risikoprofilen (f.eks. hurtigst, balanceret, tryggest) påvirker, hvordan ruteforslag rangeres. Notifikationsindstillinger styrer, hvilke typer advarsler Brugeren modtager.
F7	Se hele kortet farvekodet efter tryghed, uden at have søgt en rute	Should have	Gader og områder er farvekodet efter tryghed direkte på kortets forside, så Brugeren kan danne sig et overblik, selv uden at have søgt en konkret rute.
F8	Se detaljer om en enkelt hændelse ved tryk på dens markør på kortet	Should have	Viser en kort informationsboks med hændelsens type, tidspunkt og en eventuel beskrivelse.
F9	Se kriminalitetsrelateret nyhedsfeed for en by	Could have	Feedet viser kun kriminalitetsrelaterede nyheder for den valgte by og opdateres løbende.
F10	Vælge transportform (gang, cykel eller bil) frem for kun gang	Could have	Systemet er i denne iteration afgrænset til gang, men en transportvælger kunne udvide relevansen til flere brugergrupper.
F11	Se historik over egne tidligere indrapporteringer	Could have	Giver Brugeren mulighed for at følge status på egne rapporter, f.eks. om de er blevet bekræftet af andre.
F12	Dele en valgt rute med andre, f.eks. via et link	Could have	Kan bruges til, at en Bruger lader en anden person følge med i, hvilken rute vedkommende går.
F13	Skifte appens sprog mellem dansk og engelsk	Could have	Relevant for bl.a. turister, der ikke taler dansk.
F14	Manuel overstyring af en defineret tryghedszone, f.eks. ved en akut opstået hændelse, der endnu ikke afspejles i det automatiske datagrundlag	Won't have	Ville kræve en administratorrolle og en godkendelsesproces, som ikke indgår i denne iteration.
F15	Understøtte bilnavigation som primær rutetype	Won't have	Systemet er afgrænset til fodgængere, jf. problemformuleringen i kapitel 1.
F16	Fungere fuldt offline uden internetforbindelse	Won't have	Ville kræve lokal lagring af kort- og scoringsdata og er vurderet for omfattende til denne iteration.
F17	Direkte alarmopkald til 112 fra appen	Won't have	Kræver formelt samarbejde med alarmcentralen og ligger derfor uden for projektets rækkevidde

3.3 Ikke-funktionelle krav

3.3.1 Elicitering og analyse

De ikke-funktionelle krav er udledt af de kvalitetsegenskaber, der er afgørende for, at et system som SafeRoute reelt kan bruges og stoles på: hvor hurtigt det svarer, hvor frisk dataene er, hvor godt det forklarer sig selv, og hvor godt det beskytter de brugere, der bidrager med data.

Et konkret eksempel er kravet til svartid ved rutesøgning. SafeRoute skal kunne bruges i en mellemstor bymidte en fredag aften, hvor der i myldretiden (kl. 22-24) kan være op til 5.000 samtidige brugere. Antages det, at 2% af disse forespørger en ny rute inden for et givent minut, svarer det til:

Nbrugere = 5.000
Forespørgsler/min = 2% × 5.000 = 100
Forespørgsler/sek = 100 / 60 ≈ 1,7

For at ruteforslaget skal føles øjeblikkeligt, ligesom man kender det fra almindelige kortapps, sættes en øvre grænse for svartiden:

Trute,maks = 3 sekunder ved op til 2 forespørgsler/sekund

Et andet eksempel er kravet til dataens friskhed. Community-indrapporteringer har kun værdi, hvis de når frem til andre brugere, mens de stadig er relevante, typisk inden for den tid, det tager at gå et par gadeblokke. Det giver anledning til et konkret krav om, hvor hurtigt en ny indrapportering skal være synlig for andre.

3.3.2 Specifikation

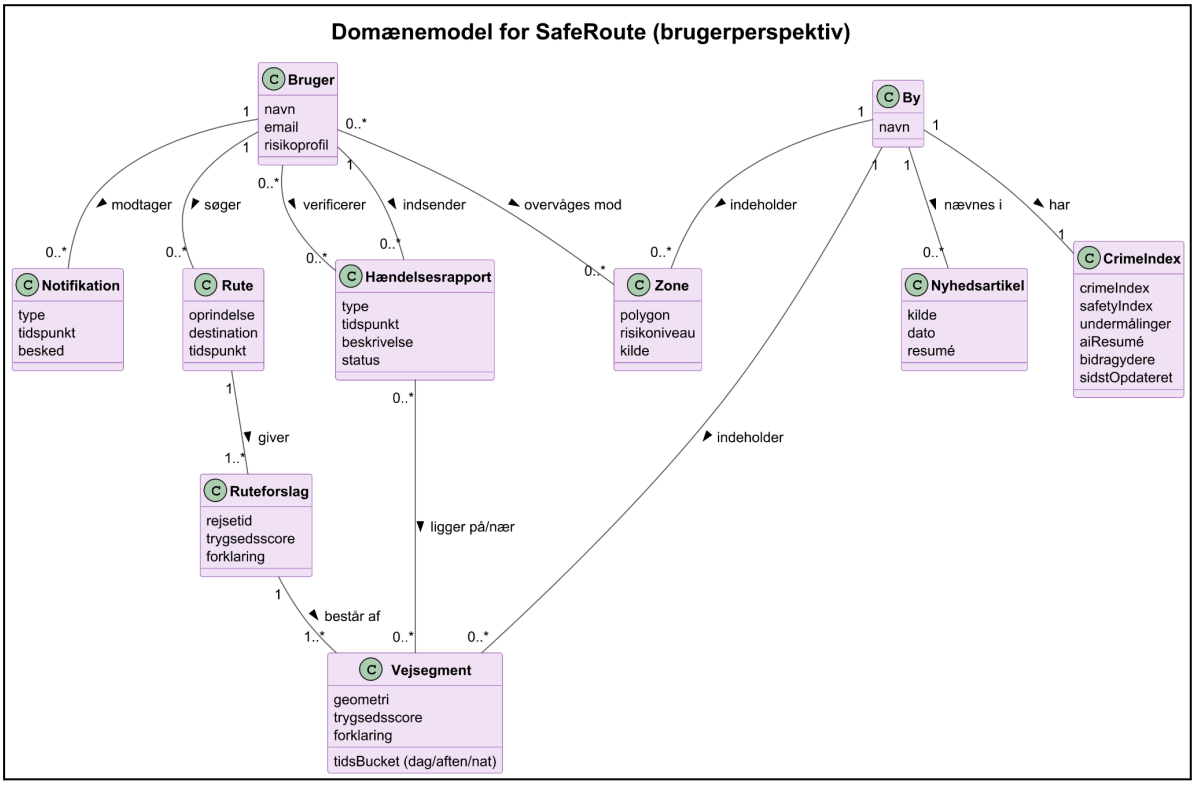
Tilsvarende er kravet til beskyttelse af bidragydernes privatliv udledt direkte af systemets afhængighed af brugerdreven data: skal brugere kunne stoles på at indrapportere, må det ikke være muligt at genkende dem ud fra de aggregerede data, systemet viser offentligt.

#	Kategori	Beskrivelse
NF1	Performance	Systemet skal kunne besvare en ruteforespørgsel, inklusive tryghedsscore for alle forslag, inden for 10 sekunder ved op til 2 samtidige forespørgsler pr. sekund.
NF2	Performance	Den stedbestemmelse, systemet baserer sig på, skal have en nøjagtighed på maksimalt 10-20 meter i et byområde, da både geofence-advarsler og tryghedscorer pr. vejsegment mister deres mening, hvis Brugerens position ikke kan bestemmes med rimelig sikkerhed.
NF3	Performance	Under en aktiv gåtur skal Brugerens position opdateres mindst hvert 15.-30. sekund, eller for hver 25 meter bevægelse, så en geofence-advarsel kan nå frem, mens Brugeren stadig er tæt på det pågældende område, og ikke først efter vedkommende er gået videre.
NF4	Pålidelighed	En ny, ubekræftet indrapportering skal være synlig på kortet for andre brugere inden for 2 minutter fra indsendelse.
NF5	Pålidelighed	En hændelse, der opnår tilstrækkelig verifikation, skal kunne påvirke den viste tryghedsscore for det pågældende område inden for 5 minutter.
NF6	Pålidelighed	Kortfunktionen (visning af ruter og tryghedsscorer) skal have en oppetid på mindst 99% målt over en kalendermåned, da systemet er tiltænkt daglig brug.
NF7	Pålidelighed	Er en enkelt ekstern datakilde (f.eks. en nyhedskilde) midlertidigt utilgængelig, skal systemet fortsat kunne vise ruteforslag og tryghedsscorer baseret på de resterende datakilder, frem for at fejle helt.
NF8	Brugervenlighed	Enhver præsenteret tryghedsscore skal ledsages af en kort, lettilgængelig tekstforklaring, der fremhæver de primære faktorer bag vurderingen (såsom belysningsniveau eller nylige rapporteringer), uden at det medfører mærkbar forsinkelse i visningen.
NF9	Brugervenlighed	En ny bruger skal kunne gennemføre sin første ruteforespørgsel, fra åbning af appen til valgt rute, uden at oprette en konto. Login skal først kræves ved indrapportering af en hændelse.
NF10	Sikkerhed og privatliv	Indrapporteringsfunktionen skal beskyttes mod misbrug og spam, så en enkelt bruger maksimalt kan indsende et fastsat antal rapporter pr. tidsenhed.
NF11	Sikkerhed og privatliv	Systemet skal garantere anonymitet for bidragydere, således at offentliggjorte tryghedsdata, markører og aggregerede kortvisninger ikke kan spores tilbage til den enkelte brugers identitet eller specifikke færden.
NF12	Kompatibilitet og skalerbarhed	Systemet skal fungere ens for Brugeren, uanset om telefonen kører iOS eller Android, så ingen af de to store brugergrupper udelukkes.

NF13	Kompatibilitet og skalerbarhed	Det skal være muligt at udvide systemet til nye byer, efterhånden som datagrundlaget bliver tilgængeligt, uden at det kræver ændringer i kernefunktionaliteten for de byer, der allerede understøttes.

3.4 Systemets domænemodel

Domænemodeller beskriver de begreber, en bruger møder: en Bruger søger Ruter, der hver giver flere Ruteforslag bestående af Vejsegmenter med hver deres tryghedsscore, tidsafhængige bucket (dag/aften/nat) og forklaring. Brugere indsender og verificerer Hændelsesrapporter, som knytter sig til vejsegmenter. En By samler sine vejsegmenter, sit CrimeIndex, sine Nyhedsartikler og sine Zoner, som Brugere kan overvåges imod i realtid (jf. F5) og modtage Notifikationer på baggrund af. Modellen forholder sig udelukkende til funktionalitet, data og systemets ydre grænser, ikke til teknologi eller interne grænseflader. Det er genstand for arkitektur- og designkapitlerne.

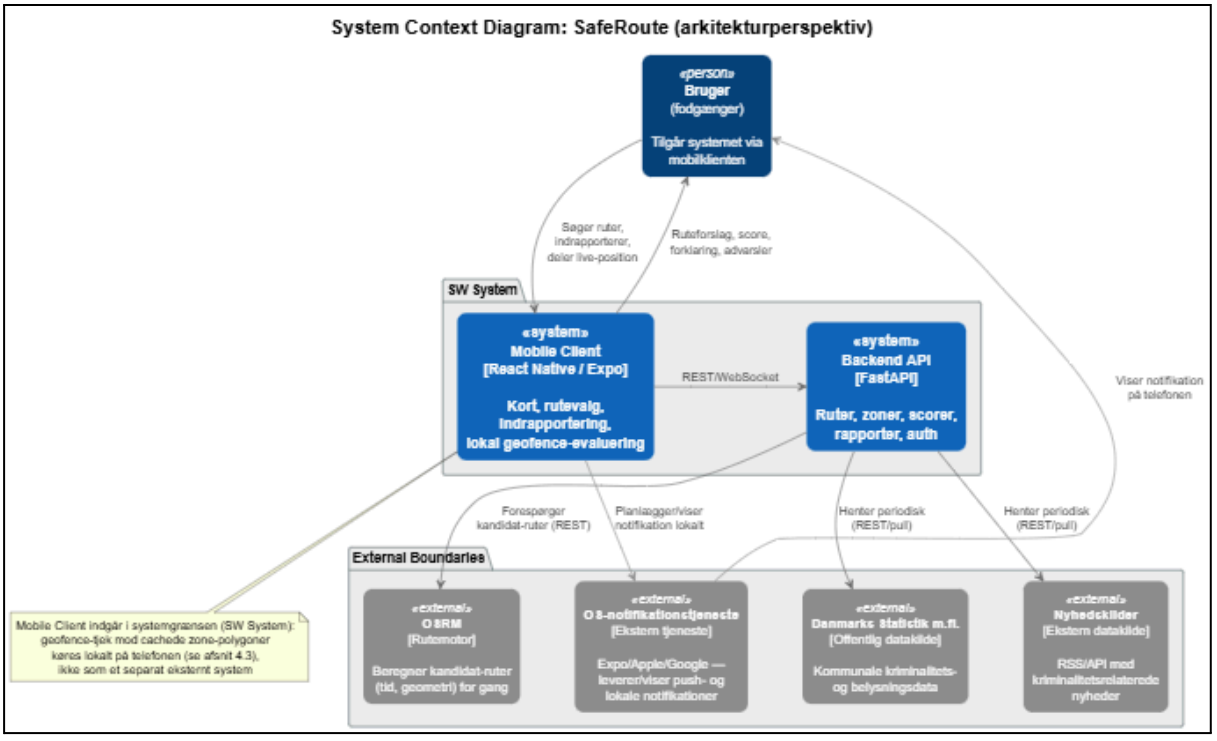


4. Systemarkitektur

4.1 Systemets ydre grænser og relationer

SafeRoute afgrænses til mobilklienten og backend'en tilsammen; det, brugeren opfatter som "appen", er begge dele. Systemet har fire eksterne relationer, som det ikke selv kontrollerer:

- **OSRM** (Open Source Routing Machine) leverer den rå ruteberegning (geometri og gangtid) mellem to punkter. SafeRoute tilføjer selv trygsedsscoren oven på OSRM's ruteforslag; OSRM ved intet om tryghed.
- **Offentlige datakilder** (bl.a. Danmarks Statistik) leverer kommunale kriminalitetsrater, som indgår i beregningen af zoner og trygshedsscorer.
- **Nyhedskilder** leverer kriminalitetsrelaterede nyheder via RSS/API til nyhedsfeedet og som et supplerende signal til scoringen.
- **OS-notifikationstjenesten** (Expo/Apple/Google's push-infrastruktur) er det lag, SafeRoute i sidste ende afsender og viser notifikationer igennem, uanset om notifikationen er udløst lokalt på telefonen eller (i et fremtidigt scenarie) sendt fra serveren.



Figur x viser systemgrænsen og disse fire relationer. Et vigtigt arkitekturvalg er allerede indlejret i figuren og uddybes i afsnit x: mobilklienten er en del af systemet, ikke en ekstern klient til en ren server-arkitektur, fordi geofence-logikken (condition'en, der udløser en push-notifikation under en gåtur) bevidst er placeret på telefonen frem for på backend'en.

4.2 Analyse af arkitektur

Den indledende, domænemodelbaserede tilgang fra kapitel 3 ("Backend gør alting, klienten viser det") holder ikke, når den holdes op mod de ikke-funktionelle krav. Risikoen er ikke isoleret til én funktion: den opstår, hver gang et NF-krav rammer et sted i arkitekturen, hvor den enkle løsning enten er for langsom, for sårbar over for en enkelt ekstern afhængighed, eller lækker mere data, end den behøver. Dette afsnit gennemgår fire arkitekturkritiske risici, der tilsammen dækker rutemotoren, realtidsadvarslerne, den community-drevne scoring og privatlivslaget, altså de fire delsystemer, der introduceres i figur 4.2, og den arkitekturelle løsning på hver.

4.2.1 Ruteberegning under belastning (NF1, NF6)

NF1 kræver, at en ruteforespørgsel, inklusive tryghedsscore for alle forslag, besvares inden for 10 sekunder ved op til 2 samtidige forespørgsler pr. sekund. De 10 sekunder er rigeligt til at dække én enkelt forespørgsels egen behandlingstid, selv med live model-inferens pr. vejsegment for hvert ruteforslag:

3 ruteforslag × ca. 100 vejsegmenter/forslag = 300 segmenter pr. forespørgsel 300 segmenter × 5 ms model-inferens/segment = 1.500 ms scoringstid pr. forespørgsel

1,5 sekund er langt under budgettet, men NF1 stiller ikke kun krav til én forespørgsels egen svartid, den stiller krav til, at systemet kan følge med op til 2 forespørgsler i sekundet, løbende. Beregnes scoringen synkront og serielt (én forespørgsel ad gangen), kan backend'en højst nå ca. 1.500 ms^{-1} = 0,67 forespørgsler i sekundet, under de 2 forespørgsler i sekundet, NF1 kræver. Ankommer forespørgslerne hurtigere, end de kan behandles, vokser køen ubegrænset, og selv en generøs 10-sekunders grænse pr. forespørgsel bliver til sidst overskredet af ventetiden i køen alene, ikke af selve beregningen: et gennemløbsproblem, ikke et enkelt-forespørgsel-latensproblem. Samtidig gør OSRM's rolle som en enkelt, ekstern rutemotor NF6's krav om 99% oppetid til en risiko, hvis backend'en afhænger direkte af én instans uden fallback.

Løsning: scoringen flyttes ud af den synkrone forespørgselsvej. Tryghedsscoren for et vejsegment genberegnes periodisk af en asynkron proces (målsat til hvert 15. minut, se Async Worker i afsnit 4.3 og 4.34) og gemmes i Datalaget, så en ruteforespørgsel kun skal slå allerede beregnede scorer op via et indekseret geospatialt opslag, ikke køre model-inferens. Et sådant opslag tager typisk under 10 ms, hvilket løfter den teoretiske gennemløbskapacitet til et par hundrede forespørgsler i sekundet, langt over de 2/s, NF1 kræver, og dermed forsvinder kørisikoen, uanset hvor tæt på grænsen budgettet sættes. NF6-risikoen ved OSRM som enkelt ekstern afhængighed reduceres ved, at Backend API falder tilbage på en lokal, forenklet ruteberegning, hvis OSRM ikke svarer (NF7), og ved at OSRM selvhostes som en container i driftsmiljøet (figur 4.3) frem for at afhænge af en offentlig demo-instans uden driftsgaranti.

4.2.2 Geofence-baseret notifikation vs. serverstyret push (NF3, NF11)

F5 kræver, at en bruger advares, både ved indtræden i et usikkert område og på tidspunkter, hvor risiko historisk stiger, mens en gåtur er i gang. NF3 stiller et konkret krav til, hvor ofte det kan opdages:

Positionsopdatering ≥ hvert 15.-30. sekund, eller for hver 25 meter bevægelse

Den oplagte arkitektur er at lade backenden modtage brugerens live-position løbende, evaluere den mod zone-polygonerne server-side, og sende en push-notifikation, når en zone krydses. Det ville imidlertid indebære, at telefonen skal streame sin position til serveren med samme frekvens som NF3 kræver, hvilket introducerer to risici:

- Latens og robusthed. Hver zonekontrol ville afhænge af en netværks-roundtrip (position op, evaluering, notifikation ned) plus serverens svartid. Falder forbindelsen ud, eller er den langsom, netop dér hvor en bruger bevæger sig gennem et område uden dækning, forsvinder advarslen i det øjeblik, den er mest nødvendig.
- Datamininering. NF11 stiller krav om, at aggregerede positions- og rapportdata anonymiseres med differential privacy, før de indgår i offentligt viste scorer. Selvom NF11 formelt kun dækker de data, der bliver offentligt synlige, ville en løbende server-side positions-stream i praksis betyde, at systemet opbygger en detaljeret, tidsstemplet bevægelseshistorik pr. bruger, præcis den type data, differential privacy-laget er designet til at undgå at eksponere. At undgå at indsamle data, man ikke har brug for at gemme, er en billigere og mere robust beskyttelse end at anonymisere det bagefter.

Løsning: geofence-evalueringen er lagt som en lokal condition på telefonen, ikke som et server-kald. Mobilklienten henter zone-polygonerne (GET /api/zones, evt. suppleret af /ws/zones) og cacher dem lokalt; Location.watchPositionAsync (interval 15 sek. / 25 m, jf. NF3 direkte) evaluerer herefter to conditions lokalt for hver ny position: (a) et point-in-polygon-tjek mod de cachede rødzoner, og (b) et tidspunkt-tjek mod en fast tidsramme. Er en condition opfyldt, planlægges notifikationen direkte via telefonens eget notifikations-API, uden en tur til backend. Dette reducerer risikoen på to måder: advarslens latens bliver

bundet af telefonens egen positionsopdatering (NF3), ikke af en netværksforbindelse, og backend modtager aldrig en løbende strøm af brugerens position under en gåtur, kun de positioner, brugeren selv aktivt sender via ruteopslag eller en hændelsesrapport. Figur 4.5 (afsnit 4.4.1) viser flowet.
Denne løsning har en bevidst begrænsning: den dækker kun conditions, klienten selv kan afgøre ud fra allerede hentede data (position, cachede polygoner, klokkeslæt). Et scenarie som "en anden bruger har lige indrapporteret og fået bekræftet en hændelse 50 meter foran dig, mens du går" kræver, at backend selv kan initiere en notifikation, hvilket er årsagen til, at "OS-notifikationstjeneste" fortsat indgår som en ekstern relation i figur 4.1 og som en planlagt server→klient-kant i containerdiagrammet (figur 4.2): den dækker den serverinitierede sti, mens den klientlokale sti dækker den almindelige, latenskritiske sti.

4.2.3 Frisk tryghedsscore fra bekræftede indrapporteringer (NF4, NF5)

F3 kræver, at en indrapportering skal bekræftes af flere nærliggende brugere, før den vægtes fuldt i trygshedsscoren. To krav rammer forskellige led i den kæde, og skal derfor ikke løses med den samme mekanisme: NF4 kræver, at en ny, ubekræftet rapport er synlig for andre brugere inden for 2 minutter; NF5 kræver, at en rapport, der opnår tilstrækkelig verifikation, kan påvirke den viste trygsedsscore inden for 5 minutter.

NF4 opfyldes af den simple del af arkitekturen: GET /api/reports læser rapporter direkte fra datalaget, så en ny rapport bliver synlig for andre klienter, næste gang de forespørger, typisk sekunder efter indsendelse.

NF5 er derimod en reel arkitektonisk risiko, hvis den holdes op mod den asynkrone scoringsproces fra det foregående punkt: tryghedsscoren genberegnes af Async Worker på en fast periode, sat til hvert 15. minut. En rapport, der lige er blevet tilstrækkeligt bekræftet, kan i værste fald vente op til 15 minutter på at slå igennem i den viste score, tre gange længere end NF5 tillader.

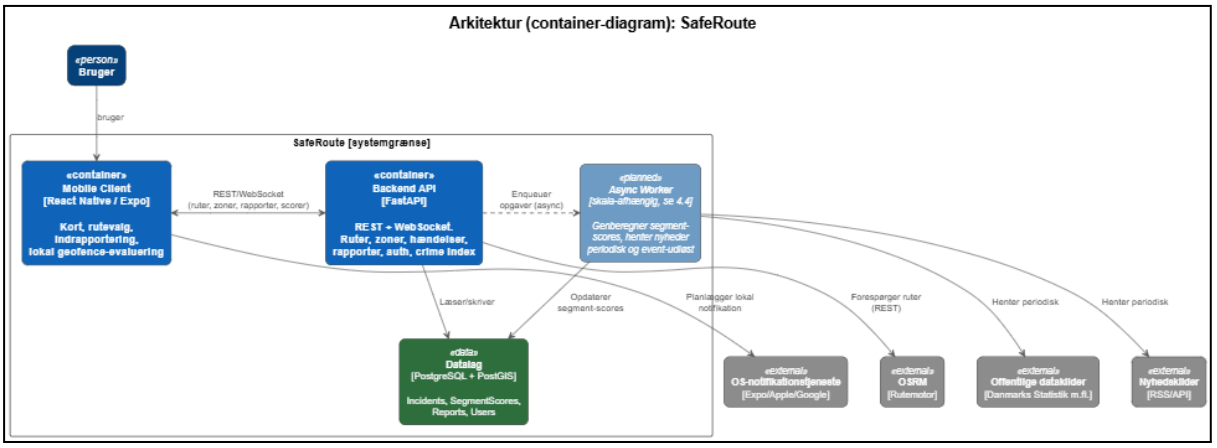
Løsning: den periodiske genberegning bevares til at indfange langsomme, akkumulerede ændringer (nye officielle data, tidspunkt-skift), men suppleres med en event-udløst genberegning: når POST /api/reports/{id}/confirm gør en rapport tilstrækkeligt verificeret, sender backend API et målrettet genberegningsjob for det berørte områdes segmenter til Async Worker, i stedet for at vente på den næste periodiske kørsel. Det holder NF5's 5-minuttersbudget uafhængigt af, hvor langt det periodiske interval er sat, uden at hver enkelt bekræftelse skal udløse en fuld genberegning af hele byen.

4.2.4 Placering af differential privacy-laget (NF11, NF13)

NF11 stiller krav om, at personoplysninger behandles i overensstemmelse med GDPR, og at aggregerede tryghedsdata anonymiseres, så den enkelte bidragyders identitet og færden ikke kan udledes. For at opfylde dette anvendes en Laplace-baseret differential privacy-mekanisme med et fastsat privatlivsbudget på $\epsilon \leq 1,0$. Selve DP-mekanismen er ukompliceret, men hvor i arkitekturen den skal kaldes, er en arkitekturkritisk beslutning i sig selv: mekanismen trækker et nyt, tilfældigt støjtal ved hvert kald. Kaldes den ved hver enkelt læsning af en offentlig score, f.eks. inde i selve GET /api/zones-endpointet, vil to på hinanden følgende opslag på samme zone returnere to forskellige støjede tal for den samme underliggende sande værdi. En bruger, der forespørger det samme endpoint mange gange, kan da gennemsnitliggøre støjen væk og tilnærme den ustøjede værdi, et klassisk "averaging attack" mod differential privacy, der reelt underminerer NF11, selvom mekanismen teknisk set anvendes.

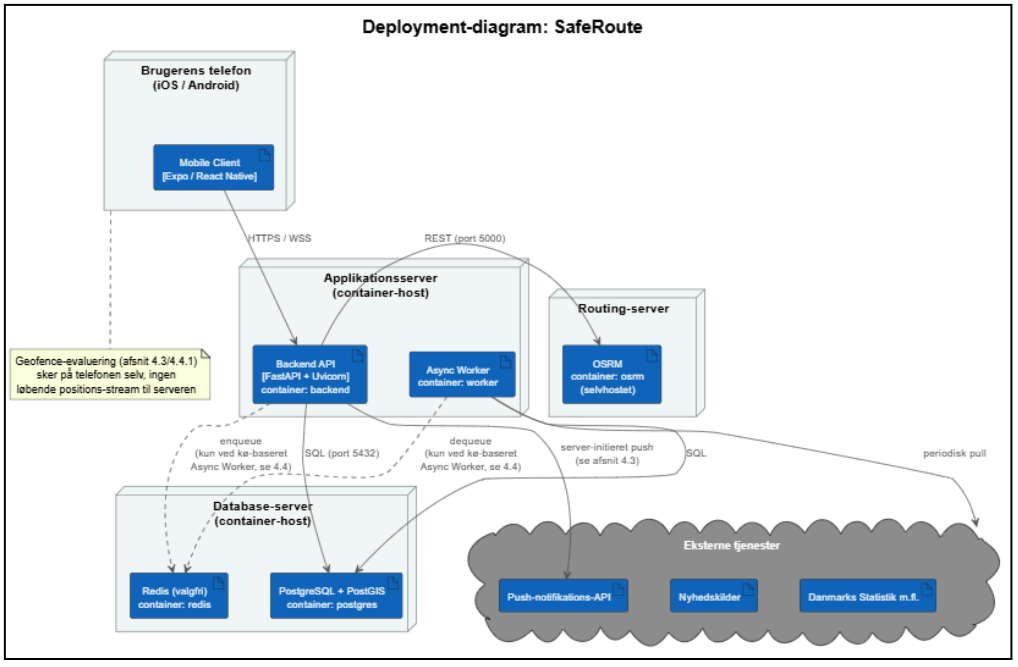
Løsning: DP-støjen skal lægges på én gang, på skrivetidspunktet, når en aggregeret statistik beregnes og caches, samme sted som den periodiske og event-udløste genberegning fra ovenstående punkt allerede skriver til datalaget. Alle efterfølgende læsninger af den offentlige API returnerer dermed den samme, allerede-støjede værdi, indtil næste genberegning, hvilket lukker averaging-angrebet uden at ændre selve DP-mekanismen. Dette optages her som en eksplicit arkitekturbeslutning, der skal respekteres, når scoringspipelinen i Async Worker implementeres, frem for at overlade placeringen til den enkelte udvikler at opdage undervejs.

4.3 Systemets struktur og grænseflader



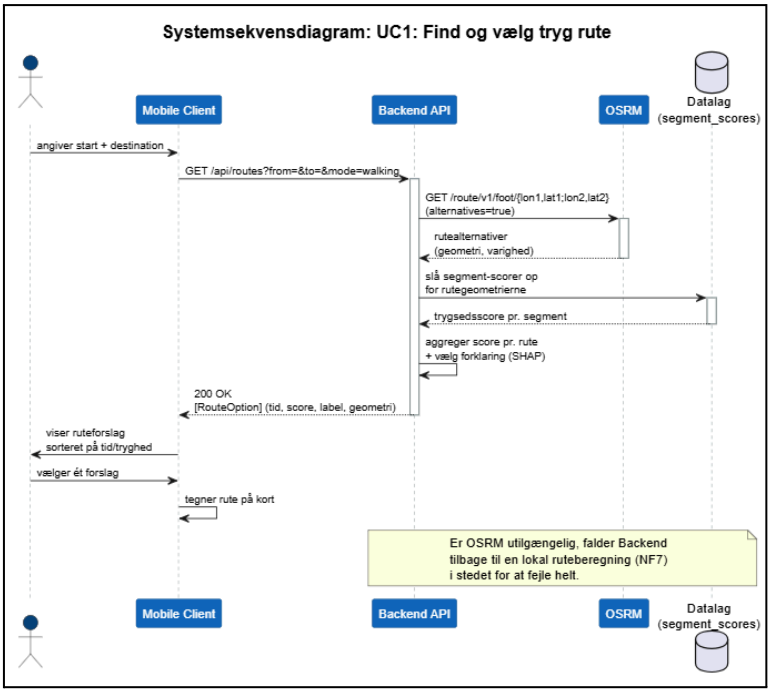
Analysen i dette afsnit giver fire delsystemer, vist som et container-diagram i figur 4.2: Mobile Client (React Native/Expo), Backend API (FastAPI, REST + WebSocket), Async Worker og et fælles Datalag (PostgreSQL/PostGIS). Mobile Client og Backend API er de eneste delsystemer, brugeren interagerer direkte eller indirekte med; Async Worker og Datalag understøtter dem.

Async Worker's realisering afhænger bevidst af belastningen, systemet reelt skal bære. Ved den skala, NF1 lægger op til (op til 2 samtidige forespørgsler i sekundet, jf. afsnit 4.2), kan de periodiske og event-udløste opgaver (genberegning af tryghedsscorer, nyhedsindsamling) afvikles som en enkelt, planlagt baggrundsproces, der kører sammen med Backend API, uden behov for en selvstændig kø-infrastruktur. Vokser belastningen, eller skal flere Backend API-instanser dele arbejdet og broadcaste zoneopdateringer til hinanden over /ws/zones, udskilles Async Worker til en kø-baseret løsning (f.eks. Celery med Redis som broker), uden at grænsefladerne til de øvrige delsystemer ændres. Ved at definere Async Worker som et selvstændigt delsystem i arkitekturen (Figur 4.2) sikres en løs kobling. Dette gør det muligt at starte med en simpel baggrundsproces og senere migrere til en distribueret kø-løsning (f.eks. Celery/Redis), uden at det kræver ændringer i grænsefladerne til systemets øvrige komponenter.

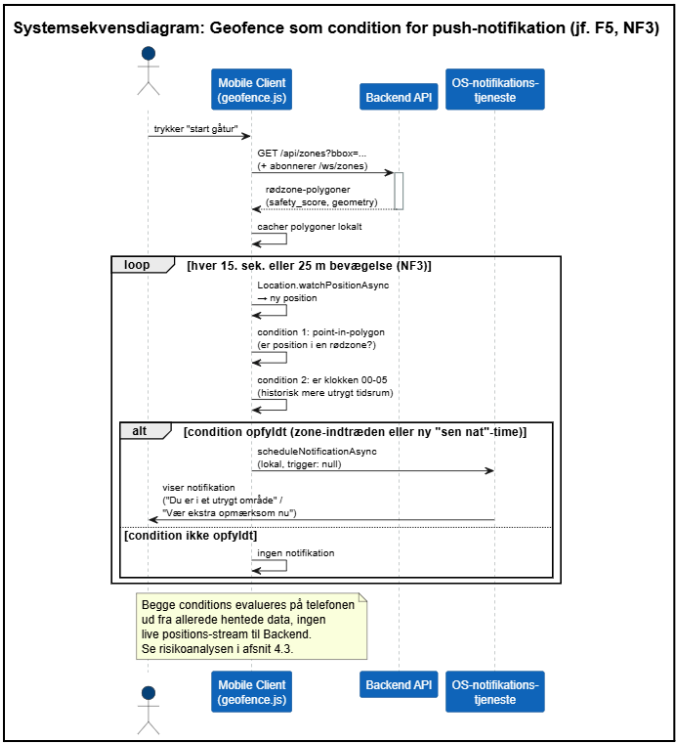


Figur 4.3 viser, hvordan delsystemerne kan allokeres til driftsmiljøet: Backend API og Async Worker deployeres som to containere på en applikationsserver, PostgreSQL/PostGIS som en container på en database-server (suppleret af Redis, hvis Async Worker udskilles til en kø-baseret løsning), OSRM som en selvstændig, selvhostet container, og Mobile Client afvikles på brugerens egen telefon.

4.3.1 Interaktion mellem delsystemer



Figur x.x viser interaktionen for UC1 (Find og vælg tryg rute, jf. kapitel 3): Mobile Client forespørger Backend API, som selv forespørger OSRM for rutegeometri og slår tryghedsscorer op i datalaget, før det samlede ruteforslag returneres.



Figur 4.5 viser interaktionen bag F5's geofence- og tidsbaserede advarsler, jf. arkitekturvalget i afsnit 4.3: Mobile Client henter zone-polygonerne én gang (eller abonnerer på ændringer via /ws/zones), hvorefter begge conditions, zoneindtræden og klokkeslæt, evalueres lokalt for hver positionsopdatering, uden yderligere kald til Backend API.

4.3.2 Interfaces og protokoller

Mobile Client - Backend API
Grænsefladen er REST over HTTP(S) med JSON, suppleret af én WebSocket-kanal (/ws/zones) til løbende zone-/scoreopdateringer. Alle skrivende kald, der kræver et login (bl.a. POST /api/reports), skal godkendes via en Authorization: Bearer <token>-header med et signeret, tidsbegrænset token (JWT), jf. NF11's krav om sikkerhed og beskyttelse af brugerdata. Tabel 4.1 opsummerer de vigtigste endpoints.

Metode	Endpoint	Formål	Krav
GET	/api/routes?from=&to=&mode=	Ruteforslag med tid + trygsedsscore	F1, NF1
GET	/api/zones?bbox=	Zone-polygoner farvekodet efter tryghed	F7
GET/POST	/api/reports	Hente/indsende hændelsesrapporter	F2
POST	/api/reports/{id}/confirm	Verificere andres rapport	F3
GET	/api/crime-index?city=	Bytryghedsoversigt	F4
GET	/api/news?city=	Kriminalitetsrelateret nyhedsfeed	F9
WS	/ws/zones	Løbende push af score-/zoneændringer	F5

Backend API - OSRM.
REST over HTTP mod OSRM's eget "Route"-endpoint (/route/v1/foot/{lon1},{lat1};{lon2},{lat2}?overview=full&geometries=geojson&alternatives=true), som returnerer GeoJSON-rutegeometri og varighed for én eller flere alternative ruter. SafeRoute tilføjer selv trygshedsscoren oven på dette svar; OSRM leverer udelukkende geometri og tid.

Backend/Worker - eksterne datakilder.
Både de offentlige kriminalitetsdata og nyhedskilderne tilgås som periodisk pull over HTTP (hhv. hver 6. time og hvert 15. minut), ikke push. Fordi disse kilder er uden for systemets kontrol, gælder NF7 direkte for denne grænseflade: er én kilde midlertidigt utilgængelig ved et opdateringsforsøg, beholder systemet det senest cachede resultat og fejler ikke ruteforespørgsler eller kortvisning som følge heraf.

Mobile Client - OS-notifikationstjeneste.
Klienten planlægger og viser lokale, øjeblikkelige notifikationer direkte via telefonens eget notifikations-API, som beskrevet i afsnit 4.3. Samme grænseflade bruges også til en serverinitieret variant (Backend sender til et push-API med et per-enhed push-token), til de conditions kun serveren kender.

4.3.3 Datamodeller og persistering

Datalaget realiseres som PostgreSQL med PostGIS-udvidelsen, så geospatiale forespørgsler (nærhedssøgning, rute-mod-polygon-forespørgsler til zoner, bbox-filtrering) kan udnytte native GEOGRAPHY-typer og GIST-indekser i stedet for at beregne dem i applikationslaget. De fire centrale entiteter er:

- **Incident**: et enkelt kriminalitets-/hændelsespunkt, tagget med kilde (official / community / news) og verifikationsstatus.
- **SegmentScore**: den forudberegnede trygsedsscore pr. vejsegment og tidsbucket (dag/aften/nat), med et felt til forklaringen (SHAP-værdier) bag scoren (NF8).
- **Report**: en brugerindsendt realtidsrapport, koblet til en bruger og med et løbende bekræftelsestal.
- **User**: kontoen, dens risikoprofil og nødkontakter.

Nyhedsartikler arkiveres separat i Datalaget, fordi RSS-feeds i sig selv ikke har nogen historik: uden en lokal arkivering ville systemet ikke kunne vise andet end de seneste ét til to dages nyheder.

4.3.4 Valg af teknologistack

Risikoanalyserne i afsnit 4.3 og strukturen ovenfor stiller en række konkrete krav til teknologien: asynkron I/O med WebSocket (NF1, /ws/zones), samme sprog som scoringsmodellen (NF8), én kodebase til iOS og Android med baggrundslokation og lokale notifikationer (NF3, NF12), native geospatiale forespørgsler (afsnit 4.4.3) og en rutemotor, systemet selv kan drifte (NF6, NF7). For hvert delsystem sammenlignes det valgte med de oplagte alternativer ud fra netop disse krav, så valget kan genbesøges, hvis forudsætningerne ændrer sig.

4.3.4.1 Backend API: FastAPI (Python)

Alternativ	Styrke	Afgørende svaghed for SafeRoute
FastAPI (valgt)	Asynkron (ASGI) fra bunden, indbygget WebSocket, automatisk validering og OpenAPI-dokumentation via Pydantic [1][2]	Pythons GIL begrænser CPU-tung parallelisme [3]; mindre "batteries included" end Django/Spring
Django + DRF	Modent ORM, admin-panel og autentificering indbygget	Async kræver ASGI, og ORM'et understøtter ikke transaktioner i async-kode; WebSocket kræver tilføjelsen Django Channels [4]
Flask	Simpelt og letvægts	WSGI: hver forespørgsel binder én worker, også ved async-views, og baggrundsopgaver kan ikke startes med asyncio.create_task [5]; ingen indbygget WebSocket eller validering
Node.js (Express/NestJS)	Stærk asynkron I/O og WebSocket-understøttelse	Scoringsmodellen (scikit-learn, SHAP) er Python; det ville kræve en ekstra Python-tjeneste og en sprogovergang mellem API og model
Spring Boot (Java)	Meget modent værktøj til sikkerhed og ORM, også geospatialt (Hibernate Spatial)	Tungere at udvikle i ved projektets skala; svagt ML-økosystem i forhold til Python

FastAPI er valgt, fordi det er det eneste alternativ, der opfylder begge de tungeste krav på én gang: asynkron håndtering af REST, WebSocket og baggrundsopgaver i samme proces, og samme sprog som scoringsmodellen, så API og scoringskode kan dele biblioteker. Node.js matcher det første krav og Django/Spring bedre det modne værktøj, men ingen af dem uden at splitte systemet i to sprog. GIL-begrænsningen er accepteret, fordi den CPU-tunge scoring alligevel er flyttet ud af forespørgselsvejen (afsnit 4.3).

4.3.4.2 Mobile Client: React Native med Expo

Alternativ	Styrke	Afgørende svaghed for SafeRoute
React Native + Expo (valgt)	Én JavaScript-kodebase til iOS og Android; færdige moduler til lokation i baggrunden og geofencing (expo-location) [6], planlagte lokale notifikationer (expo-notifications) [7] og sikker token-lagring (expo-secure-store)	Mindre kontrol over native kodestier; det administrerede workflow kan halte efter de nyeste platforms-API'er
Flutter (Dart)	Én kodebase, god rendering-performance	Nyt sprog (Dart) for teamet; lokation og geofencing afhænger af tredjeparts-plugins frem for en samlet, administreret SDK
Native (Swift + Kotlin)	Fuld kontrol over baggrundslokation og notifikationer	To kodebaser at udvikle og vedligeholde; direkte i modstrid med NF12 ved projektets ressourcer
Progressiv webapp (PWA)	Ingen app-butik, én webkodebase	Geolocation API'et leverer kun positioner, mens siden er synlig [8], og web-push på iOS kræver iOS 16.4+ og at appen er føjet til hjemmeskærmen [9]; geofence-advarslen under en gåtur (F5, NF3) kan derfor ikke realiseres

React Native med Expo er valgt, fordi den klientlokale geofence-arkitektur fra afsnit 4.3 står og falder med pålidelig lokationsovervågning og lokale notifikationer på begge platforme. Det udelukker en PWA, og native udvikling er fravalgt på grund af NF12's dobbelte vedligeholdelsesbyrde. Mellem Flutter og React Native er udslaget, at Expo leverer de præcise moduler, arkitekturen bruger, som én samlet og versionsafstemt SDK, og at JavaScript/React er samme værktøj, som også gør en webversion mulig (react-native-web).

4.3.4.3 Datalag: PostgreSQL med PostGIS

Alternativ	Styrke	Afgørende svaghed for SafeRoute
PostgreSQL + PostGIS (valgt)	Bredeste udvalg af geospatiale funktioner (GEOGRAPHY, ST_Intersects, ST_DWithin) med GIST-indeks [10]; fuld relationel model og transaktioner	Kræver indeksering og tuning; administreret hosting med PostGIS er mindre udbredt og dyrere
MongoDB	2dsphere-indeks og fire geo-operatorer ($geoWithin, $geoIntersects, $near, $nearSphere) [11]	Svagere til de relationelle data (bruger → rapporter → bekræftelser) og snævrere geospatialt funktionsudvalg
MySQL (spatial)	Udbredt; har spatial-typer og spatiale indeks [12]	Færre geospatiale funktioner end PostGIS [10][12]
SQLite (evt. SpatiaLite)	Ingen server, én fil, nul drift	Kun én skriver ad gangen; dokumentationen anbefaler selv en klient/server-database ved mange samtidige skrivere [13], hvilket rammer samtidige rapporter fra mange brugere (NF4)

PostgreSQL/PostGIS er valgt, fordi datamodellen i afsnit 4.4.3 er både relationel (brugere, rapporter, bekræftelser) og geospatial (segmenter, zoner, nærhed), og PostGIS er det eneste alternativ, der dækker begge fuldt ud. Det gør det muligt at lave rute-mod-polygon- og nærhedsforespørgsler i databasen med indeks i stedet for i applikationslaget, hvilket er forudsætningen for de hurtige opslag under 10 ms i afsnit 4.3.

4.3.4.4 Rutemotor: OSRM (selvhostet)

Alternativ	Styrke	Afgørende svaghed for SafeRoute
OSRM (valgt)	Meget hurtige forespørgsler takket være en forudberegnet graf (Contraction Hierarchies) [14][15], simpelt HTTP-API med alternative ruter (alternatives=true) [16], open source og selvhostbar	Vægte fastlægges ved forudberegningen, så tryghed kan ikke indgå direkte i ruteomkostningen; grafen skal genopbygges fra OSM-udtræk
GraphHopper	Open source, gå-profil, "custom models" kan vægte ruter pr. forespørgsel [17]	Java-tjeneste med højere hukommelsesforbrug; fleksibiliteten er ikke nødvendig, så længe scoringen lægges oven på ruteforslagene
Valhalla	Open source, dynamisk omkostningsfunktion ved forespørgselstidspunktet [18]	Mere kompleks opsætning og tuning end OSRM
Google / Mapbox Directions API	Ingen drift, høj kvalitet og dækning	Betaling pr. forespørgsel [19], ratebegrænsninger og licensvilkår for lagring/kombination af data; systemet bliver afhængigt af én ekstern udbyder (NF6)

OSRM er valgt, fordi arkitekturen bevidst adskiller ruteberegning fra trygsedsscoring: OSRM skal kun levere et par hurtige ruteforslag, som Backend API derefter scorer ud fra de forudberegnede segmentscorer (afsnit 4.3). Til den opgave er OSRM den hurtigste og simpleste selvhostbare motor, og selvhosting fjerner den afhængighed af en kommerciel udbyder, som NF6 og NF7 gør til en risiko. Ønsker man senere, at tryghed skal påvirke selve rutevalget, og ikke kun rangeringen af forslag, er GraphHopper eller Valhalla det naturlige skift, fordi de kan vægte ruter pr. forespørgsel.

4.3.4.5 Async Worker: enkelt baggrundsproces frem for Celery/Redis ved lancering

Alternativ	Styrke	Afgørende svaghed for SafeRoute
Enkelt planlagt baggrundsproces (valgt ved lancering)	Ingen broker at deploye; få bevægelige dele	Skalerer ikke ud over én proces; ingen automatisk genkørsel af afbrudte jobs
Celery + Redis (målarkitektur ved vækst)	Horisontal skalering, genkørsel af jobs, planlægning (Celery Beat) [20]; Redis kan også bruges til broadcast mellem API-instanser på /ws/zones	Ekstra infrastruktur at drifte og overvåge, som ikke er nødvendig ved NF1's belastning
RQ / APScheduler	Enklere end Celery	RQ kræver stadig Redis; APScheduler giver ingen fordeling på tværs af processer
Cron-job	Simpelt og velkendt	Kan ikke modtage de event-udløste genberegningsjobs fra POST /api/reports/{id}/confirm (NF5)

En enkelt baggrundsproces er valgt ved lancering, fordi den dækker både de periodiske og de event-udløste jobs (afsnit 4.3) ved den belastning, NF1 lægger op til, uden ekstra infrastruktur. Celery med Redis er den planlagte realisering, når belastningen eller antallet af Backend API-instanser vokser, og skiftet kan ske uden at ændre grænsefladerne til de øvrige delsystemer (afsnit 4.4).

5. Design og implementering

5.1 Introduktion

Dette kapitel beskriver design og implementering af de fire delsystemer fra arkitekturen i kapitel 4 (figur 4.2): Backend API, Async Worker, Datalag og Mobile Client. Kapitlet følger en top-down-tilgang: hvert delsystem præsenteres først som en helhed (struktur og intern interaktion), hvorefter de centrale komponenter åbnes op enkeltvis. Hvert delsystem beskrives udelukkende ud fra sine egne komponenter; de øvrige delsystemer kendes kun gennem grænsefladerne fra afsnit 4.4.2.

Kapitel 4 beskrev den planlagte arkitektur. Den nuværende implementering realiserer den samme opdeling og de samme grænseflader, men bruger bevidst forenklede udgaver af to delsystemer, så hele systemet kan køres på én udviklermaskine uden Docker. Tabel 5.1 opsummerer forskellen og danner rammen for resten af kapitlet.

Delsystem	Planlagt (kapitel 4)	Nuværende implementering	Konsekvens
Backend API	FastAPI, REST + WebSocket	FastAPI, 8 routere + /ws/zones	Som planlagt; tre endpoints er stadig stubs (afsnit 5.2)
Async Worker	Enkelt baggrundsproces → Celery/Redis ved vækst	To asyncio-løkker startet i FastAPI's lifespan (samme proces)	Svarer til lanceringsvarianten fra 4.4.4; Celery-opgaverne findes som stubs
Datalag	PostgreSQL + PostGIS	To SQLite-filer + en in-memory cache; PostGIS-skemaet findes som ORM-modeller og Alembic-migration	Ingen geospatiale indeks; ingen segment_scores endnu (afsnit 5.4)
Mobile Client	React Native + Expo	React Native + Expo, 4 faner	Geofence-modulet er implementeret men endnu ikke koblet på (afsnit 5.5)

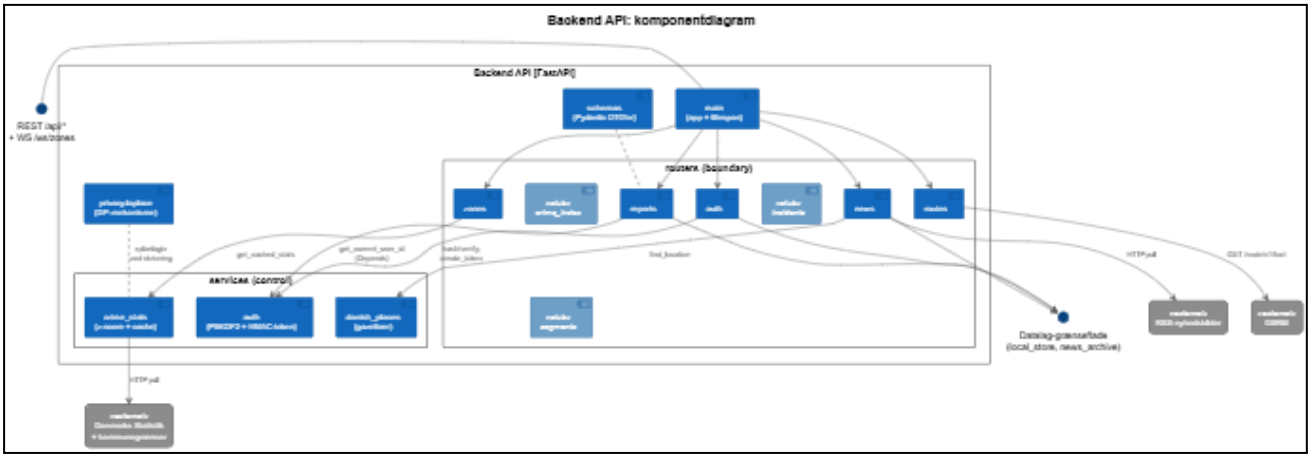
Rutemotoren (OSRM) er i udviklingsmiljøet ikke selvhostet som planlagt i figur 4.3; OSRM_SERVER_URL peger på den offentlige instans routing.openstreetmap.de/routed-foot. Grænsefladen er den samme, så skiftet til en selvhostet container kræver kun en ændring af miljøvariablen.

5.2 Delsystem Backend API

5.2.1 Introduktion

Backend API har ansvaret for at besvare alle forespørgsler fra Mobile Client: ruteforslag med trygsedsscore (F1), zone-polygoner (F7), indrapportering og verificering af hændelser (F2, F3), nyhedsfeed (F9), bytryghedsoversigt (F4) samt login. Delsystemet er skrevet i Python 3 med FastAPI og afvikles af Uvicorn.

5.2.1.1 Struktur af delsystem

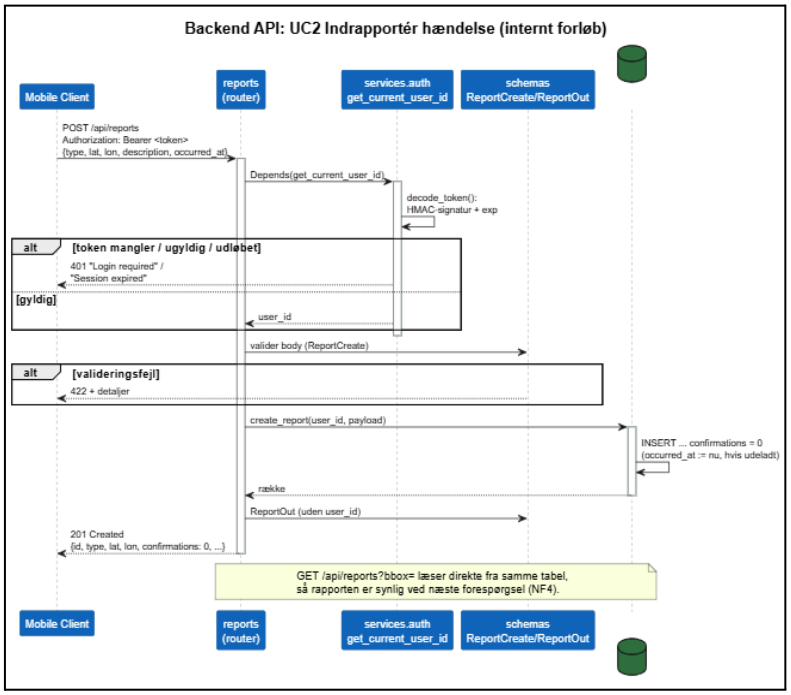


Backend API er opdelt i tre lag, som vist i komponentdiagrammet i figur 5.1:

- **Routers** (boundary): én FastAPI-APIRouter pr. ressource (auth, reports, routes, zones, news, crime_index, incidents, segments). En router oversætter HTTP til funktionskald og validerer ind- og uddata, men indeholder så lidt forretningslogik som muligt.
- **Services** (control): genbrugelig logik, som flere routere eller Async Worker deler: auth (adgangskoder og tokens), crime_stats (kommunal kriminalitetsscore og cache) og danish_places (stednavne-opslag til nyheder).
- **Schemas**: Pydantic-modeller (RouteOption, ReportCreate, ReportOut, ZoneOut, NewsItem m.fl.), der definerer kontrakten i tabel 4.1. FastAPI validerer automatisk alle requests mod dem og returnerer 422 ved fejl, og de bruges samtidig til at generere OpenAPI-dokumentationen.
- **Datalaget** tilgås kun gennem modulerne local_store og news_archive (afsnit 5.4), så routerne ikke kender til SQL eller filplaceringer. main.py samler routerne, slår CORS til og starter Async Worker's løkker i applikationens lifespan.

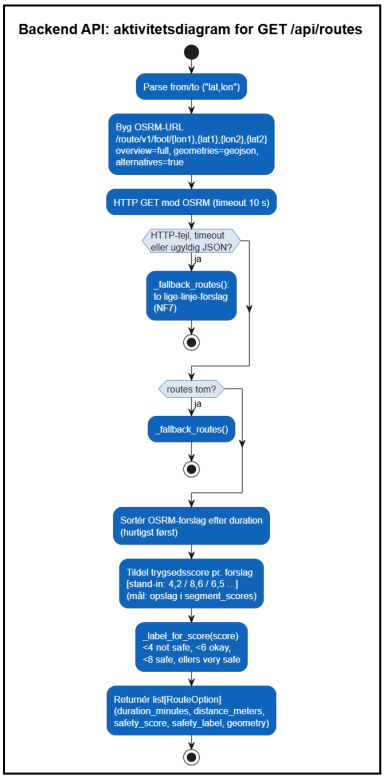
Opdelingen i routers og services er valgt, fordi den samme logik skal kunne kaldes fra flere steder: auth.get_current_user_id bruges som FastAPI-dependency af både POST /api/reports og POST /api/reports/{id}/confirm, og news.poll_and_archive_once kaldes både af Async Worker og indirekte af GET /api/news. Det giver høj samhørighed i hvert modul og lav kobling mellem routerne indbyrdes; ingen router importerer en anden router.

5.2.1.2 Interaktion mellem komponenter



Figur 5.2 viser det interne forløb for UC2 (Indrapportér hændelse). Autentificeringen er placeret som en FastAPI-dependency (Depends(get_current_user_id)) og ikke inde i routerfunktionen. Dermed afvises en request uden gyldigt token med 401, før routerens egen kode overhovedet køres, og kravet om login (F2, UC2 udvidelse 3a) kan ikke glemmes i et nyt endpoint, så længe dependency'en er erklæret i signaturen. ReportOut indeholder bevidst ikke user_id, så hvem der har indsendt en rapport aldrig sendes videre til andre brugere.

5.2.2 Design og implementering af rutetjenesten (routers/routes.py)



Rutetjenesten realiserer Backend API's del af UC1. Logikken er vist som aktivitetsdiagram i figur 5.3, da den er et lineært dataflow med to fejlgrene frem for et objektsamspil.

Endpointet modtager from og to som "lat,lon" og kalder OSRM's Route-service med alternatives=true, så OSRM leverer op til tre alternative gangruter. Bemærk, at OSRM forventer koordinaterne i rækkefølgen lon,lat; byttet sker ét sted, i URL-opbygningen. Kaldet er asynkront (httpx.AsyncClient, timeout 10 s), så en langsom OSRM ikke blokerer andre forespørgsler i samme proces, hvilket understøtter gennemløbskravet i NF1.

Robusthedskravet NF7 er implementeret ved, at enhver HTTP-fejl, timeout, ugyldig JSON eller et tomt routes-svar fører til _fallback_routes(), som returnerer to lige-linje-forslag i stedet for en fejl. Brugeren får dermed altid et svar, også når rutemotoren er nede.

Trygsedsscoren er p.t. en stand-in. Den planlagte løsning fra afsnit 4.3 er at slå forudberegnede scorer op i segment_scores for de vejsegmenter, ruten passerer. Den tabel er endnu ikke befolket (afsnit 5.4), så forslagene sorteres i stedet efter varighed, og det hurtigste tildeles 4,2, det næste 8,6 og resten 6,5. Det er en bevidst pladsholder, der bevarer UI'ets "hurtig vs. tryg"-afvejning og den fulde kontrakt (RouteOption), så Mobile Client ikke skal ændres, når den rigtige scoring kobles på. Scoren oversættes til en tekstlabel med faste grænser: under 4 "not safe", under 6 "okay", under 8 "safe", ellers "very safe".

5.2.3 Design og implementering af autentificering (services/auth.py)

Kapitel 4 krævede signerede, tidsbegrænsede tokens i en Authorization: Bearer-header. Det er implementeret uden eksterne biblioteker, med Pythons standardbibliotek:

Adgangskoder hashes med PBKDF2-HMAC-SHA256, 200.000 iterationer og et tilfældigt 16-byte salt pr. bruger, gemt som salt$digest. Sammenligningen bruger hmac.compare_digest, som tager konstant tid og derfor ikke lækker information via timing.

Tokens består af en base64url-kodet JSON-payload {"sub": user_id, "exp": ...} og en HMAC-SHA256-signatur over payloaden med JWT_SECRET_KEY. Formatet ligner en JWT, men er ikke en standard-JWT (der er ingen header-del). Levetiden er 30 dage, fordi der ikke er noget refresh-flow.

get_current_user_id afviser manglende header, forkert skema, ugyldig signatur og udløbet token med 401.

Valget af standardbiblioteket frem for f.eks. bcrypt og PyJWT følger samme princip som resten af udviklingsopsætningen: færrest mulige afhængigheder. Det er et kendt kompromis: skal tokens kunne valideres af andre tjenester, bør formatet skiftes til en rigtig JWT, og JWT_SECRET_KEY skal sættes i miljøet, da standardværdien kun er beregnet til udvikling.

Mobile Client gemmer token og bruger i expo-secure-store (afsnit 5.5), så login overlever en genstart af appen.

5.2.4 Design og implementering af zoner (routers/zones.py + services/crime_stats.py)

Zonerne til F7 og geofencing (F5) er det eneste sted i systemet, hvor rigtige kriminalitetsdata allerede driver en score. Officielle danske data findes ikke med koordinater pr. hændelse, kun som optællinger pr. kommune, så en zone er en kommunepolygon.
Scoren beregnes i tre trin:

Antal voldsforbrydelser (Danmarks Statistik, tabel STRAF11, kode 12) summeres over de seneste fire kvartaler, så små kommuner ikke domineres af kvartalsvis støj.

Raten pr. 100.000 indbyggere beregnes med befolkningstallet fra FOLK1A.

Raten omregnes til en score med en z-score i forhold til landsgennemsnittet:

```
z     = (rate − gennemsnit) / standardafvigelse
score = clamp(5,5 − 1,5 · z ; 1 ; 10)
```

En z-score er valgt frem for percentilrangering, fordi percentiler altid ville farve en fast andel af landet "utryg", uanset hvor lille den reelle spredning er. Med z-scoren lander en gennemsnitlig kommune på 5,5, og kun statistisk afvigende kommuner bliver markeret. Labelgrænserne er 3,6 ("unsafe", ca. +1,25 standardafvigelser) og 4,4 ("mixed", ca. +0,75 standardafvigelser); resten er "safe".

GET /api/zones?bbox= filtrerer de cachede polygoner med en simpel bounding-box-overlapstest. Kommuner med ø er kommer tilbage som flere polygoner med samme navn, så hvert ekstra stykke får et suffiks (samsø-2), da Mobile Client bruger id'et som React-nøgle. Er cachen tom, fordi kilden var utilgængelig, returneres en tom liste frem for pladsholderdata.

/ws/zones er stadig en stub, der sender syntetiske opdateringer hvert 5. sekund; den rigtige udgave skal publicere ændringer fra Async Worker.

5.2.5 Design og implementering af differential privacy (privacy/laplace.py)

Laplace-mekanismen til NF11 er implementeret: add_dp_noise(value, sensitivity, epsilon) lægger støj fra Laplace(0, sensitivity/ε) på en aggregeret værdi og klipper til ≥ 0, da en optælling ikke må blive negativ. privatize_grid_counts anvender den på en hel tabel af celleoptællinger, og ε læses fra settings.dp_epsilon (1,0, jf. NF11). Modulet kaldes endnu ikke af nogen endpoint. Det skal kaldes fra Async Worker's scoringspipeline, på det tidspunkt hvor aggregatet skrives, jf. arkitekturbeslutningen i afsnit 4.3, og ikke ved hver læsning (averaging-angreb).

5.2.6 Øvrige endpoints

GET /api/crime-index (F4) og GET /api/segments/{id}/explain (NF8) returnerer faste, repræsentative værdier, så Mobile Client's skærme kan udvikles mod den endelige kontrakt. GET /api/incidents returnerer en tom liste, indtil der findes en rigtig hændelseskilde. Det er valgt frem for at vise opdigtede hændelser på kortet.

5.2.7 Tests og resultater

Backend API testes med pytest og FastAPI's TestClient (mappen backend/tests/). Alle tests isoleres fra udviklingsmiljøet: begge SQLite-stores omdirigeres til midlertidige filer pr. test, lifespan køres ikke (så baggrundsløkkerne ikke rammer netværket), og OSRM erstattes af en httpx.MockTransport.
Tabel 5.2: Udvalgte enhedstests i Backend API

Komponent	Test	Forventet resultat	Status
services.auth	Adgangskode hash > verify	Rigtig accepteres, forkert afvises	
services.auth	Samme adgangskode hashes to gange	Forskellige hashes (unikt salt)	
services.auth	Token med ændret signatur	Afvises	
services.auth	Udløbet token	Afvises	
services.auth	Manglende/forkert Authorization-header	401	
routes	Labelgrænser 1 / 3,99 / 4 / 6 / 8 / 10	Korrekt label pr. grænse	
crime_stats	Gennemsnitlig kommune	Score 5,5	
crime_stats	Ekstrem outlier	Score klippes til [1; 10]	

zones	Labelgrænser 3,5 / 3,6 / 4,3 / 4,4	unsafe / mixed / mixed / safe
privacy.laplace	ε = 0	ValueError
privacy.laplace	200.000 stikprøver, b = 1	Middel ≈ 0, varians ≈ $2b^2 = 2$ (±3%)
privacy.laplace	Støj på optælling 0 (1.000 gange)	Aldrig negativ

Interface	Test	Status
auth-router → services.auth → local_store	Signup returnerer 201 + token; samme email igen giver 409; email normaliseres til små bogstaver	
auth-router → local_store	Login med forkert adgangskode 401, rigtig 200	
reports-router → services.auth	Rapport uden token 401	
reports-router → local_store	Ny rapport 201 med confirmations = 0 og synlig i GET /api/reports for bbox'en (NF4)	
reports-router → local_store	Rapport uden for bbox returneres ikke	
reports-router → schemas	user_id indgår hverken i oprettelses- eller listesvar	
reports-router → local_store	Bekræftelse øger confirmations med 1; ukendt id giver 404; uden token 401	
routes-router → OSRM (mock)	URL indeholder lon,lat-rækkefølge og alternatives=true; svar sorteres hurtigst først	
routes-router → OSRM (mock)	OSRM utilgængelig eller tomt svar → fallback, 200 med 2 forslag (NF7)	
zones-router → crime_stats	Bbox-filtrering og unikke id'er for ø-stykker; tom cache → tom liste	

Mangler tekst

Modul	Line coverage
routers/auth.py, routers/reports.py, routers/routes.py	
services/local_store.py, privacy/laplace.py	
services/auth.py	
services/news_archive.py	
routers/zones.py	

Hele app/

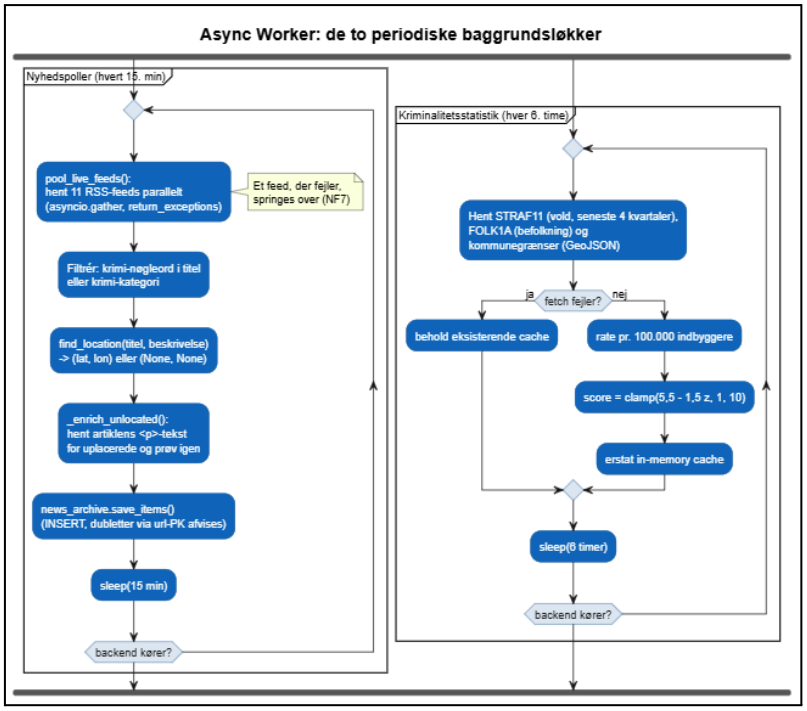
5.2.8 Delkonklusion

5.3 Delsystem Async Worker

5.3.1 Introduktion

Async Worker har ansvaret for de periodiske opgaver, der holder data friske uden at belaste forespørgselsvejen: indsamling af kriminalitetsnyheder og genberegning af zonescorer. I overensstemmelse med lanceringsvarianten i afsnit 4.4.4 er den realiseret som en enkelt, tidsstyret baggrundsproces, nærmere bestemt to asyncio-tasks, som startes i FastAPI's lifespan og stoppes, når applikationen lukker.

5.3.2 Struktur og interaktion



Figur 5.4 viser de to løkker som et aktivitetsdiagram, da Async Worker er et rent dataflow uden tilstand mellem kørslerne udover hvad der skrives til Datalaget.

Nyhedspolleren kører hvert 15. minut: henter 11 danske RSS-feeds (DR, Ekstra Bladet, Politiken, Berlingske og syv TV2-regionalstationer) parallelt med asyncio.gather(..., return_exceptions=True), filtrerer på krimi-nøgleord eller -kategorier, placerer hver nyhed geografisk og arkiverer den.

Statistikløkken kører hver 6. time og genopbygger zone-cachen i crime_stats (afsnit 5.2.4). Kildedataene opdateres kun kvartalsvis, så intervallet er rigeligt.

Begge løkker fanger alle undtagelser, logger dem og prøver igen ved næste interval, så en fejlende ekstern kilde aldrig stopper backend'en (NF7). Parallelhentningen med return_exceptions=True betyder desuden, at ét utilgængeligt feed kun fjerner sine egne nyheder fra kørslen.

5.3.3 Design og implementering af nyhedsindsamlingen

RSS-feeds rummer kun de seneste 1-2 dages nyheder. For at opbygge historik gemmer polleren hver ny nyhed i news_archive (afsnit 5.4), hvor URL'en er primærnøgle, så de samme nyheder, der serveres ved hver polling, ikke dubleres. Historiske nyheder fra 2020 og frem kan desuden importeres én gang via POST /api/news/backfill, som læser URL-indekset fra Internet Archive's Wayback Machine og udleder en overskrift af URL'ens slug, uden at hente artiklernes indhold.

Nyhederne indeholder ingen koordinater. danish_places.find_location søger derfor titel og beskrivelse igennem efter ét af 135 danske by-, bydels- og kommunenavne (ordgrænse-regex, kompileret én gang). Findes intet sted, henter polleren artiklens <p>-afsnit og prøver igen. Kun <p>-indhold bruges, fordi menuer, footere og "relaterede artikler" ofte nævner andre byer og ellers giver forkerte placeringer. Berigelsen sker kun i baggrundsløkken, aldrig i det interaktive GET /api/news, så brugeren ikke venter på den.

Celery-opgaverne recalculate_segment_scores og ingest_news (tasks.py, med Beat-skema hvert 15./30. minut) er bevaret som stubs og beskriver målarkitekturen: hent hændelser, byg features (ml/features.py), kør modellen med SHAP (ml/serving.py), læg DP-støj på og skriv til segment_scores. Modellen (GradientBoostingRegressor) kan trænes end-to-end med ml/train.py, men kun på syntetiske data.

5.3.4 Tests og resultater

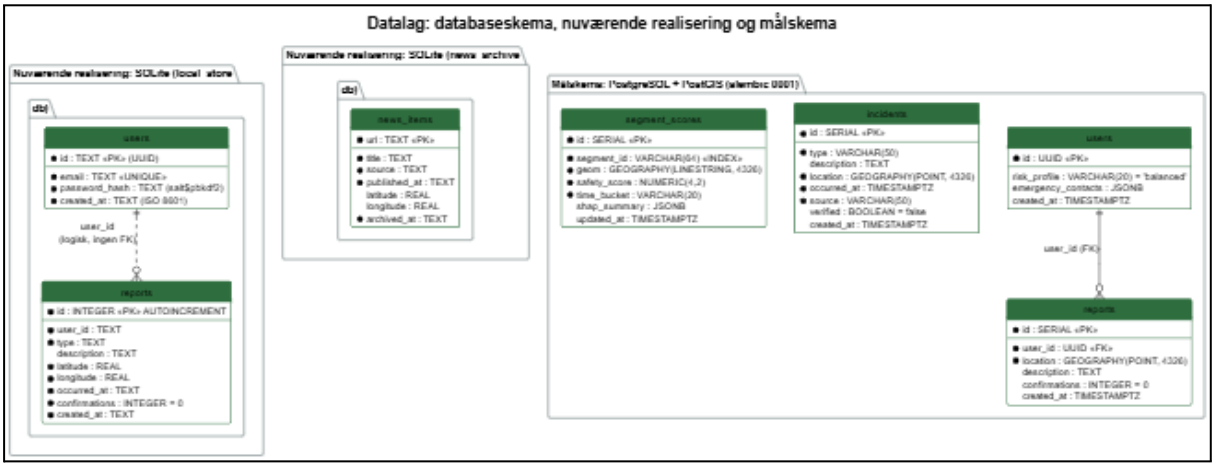
Test	Forventet resultat	Status
Krimi-filter: nøgleord i titel	Medtages	
Krimi-filter: kategori "Krimi" uden nøgleord	Medtages	
Krimi-filter: ikke-krimi titel og kategori	Frasorteres	
find_location med bynavn / uden stednavn	Byens koordinat / (None, None)	
Bydel vinder over by ("Nørrebro i København")	Nørrebros koordinat	
Arkivering af samme nyhed to gange	1 ny, derefter 0	
Årsfilter i arkivet	Kun nyheder fra det valgte år	

5.3.5 Delkonklusion

5.4 Delsystem Datalag

5.4.1 Introduktion

Datalaget har ansvaret for at persistere brugere, indrapporteringer, nyhedsarkivet og, i målarkitekturen, hændelser og forudberegnede segmentscorer. Figur 5.5 viser både den nuværende realisering og målskemaet.



5.4.2 Design og implementering af nuværende realisering (SQLite)

Datalaget er p.t. realiseret som to SQLite-filer og en in-memory cache:

local_store.db: users (UUID som tekst-PK, unik email, password_hash) og reports (auto-increment-PK, type, latitude/longitude som REAL, occurred_at, confirmations med standardværdi 0). user_id er en logisk reference uden fremmednøgle-constraint.

news_archive.db: news_items med url som primærnøgle. PK'en er selve dubletkontrollen: INSERT af en kendt URL fejler med IntegrityError, som fanges og tælles som "ikke ny".

In-memory cache i crime_stats for kommunepolygoner og scorer, da de kan genberegnes fra kilden og kun ændres kvartalsvis.

Tabellerne oprettes idempotent med CREATE TABLE IF NOT EXISTS ved opstart. Hver operation åbner og lukker sin egen forbindelse via en context manager, der committer ved succes, så der ikke deles forbindelser mellem tråde. Tidsstempler gemmes som ISO 8601-tekst, som sorterer korrekt alfabetisk; det udnyttes af årsfilteret i arkivet (published_at >= 'ÅÅÅÅ-01-01' AND < 'ÅÅÅÅ+1-01-01') uden at hver række skal parses.

Bbox-forespørgslen på rapporter er et BETWEEN på longitude og latitude uden indeks, altså en fuld tabelscanning. Det er acceptabelt ved prototypens datamængde, men er netop den forespørgsel, der skal flyttes til et GIST-indeks i målskemaet.

SQLite er valgt i udviklingsfasen, fordi det ikke kræver en databaseserver. Afsnit 4.4.4 fravalgte SQLite til drift, fordi den kun tillader én skriver ad gangen; det gælder fortsat.

5.4.3 Design af målskema (PostgreSQL + PostGIS)

Målskemaet findes som SQLAlchemy/GeoAlchemy2-modeller (app/models.py) og som Alembic-migrationen 0001_initial_schema, der opretter PostGIS-udvidelsen og tabellerne users, incidents, segment_scores og reports. Positioner er GEOGRAPHY(POINT, 4326) og vejsegmenter GEOGRAPHY(LINESTRING, 4326), så afstande beregnes i meter på jordkuglen. segment_scores har et B-tree-indeks på segment_id, og shap_summary er JSONB til forklaringen bag scoren (NF8). reports.user_id er en rigtig fremmednøgle til users.
Før skiftet fra SQLite kræves en migration 0002, fordi de to skemaer er gledet fra hinanden under udviklingen:

- users mangler email og password_hash (login blev tilføjet efter migrationen).
- reports mangler type og occurred_at.
- GIST-indeks på location og geom bør erklæres eksplicit i migrationen, så bbox- og rute-mod-segment-opslagene kan køre som indekserede forespørgsler (ST_Intersects, ST_DWithin), jf. målet på under 10 ms i afsnit 4.3.

5.4.4 Tests og resultater

Datalaget testes gennem integrationstestene i tabel 5.3 og arkivtestene i afsnit 5.3.4, mod midlertidige SQLite-filer. local_store.py har 100% og news_archive.py 95% line coverage. Målskemaet er ikke testet, da det ikke køres i udviklingsmiljøet; CI-workflowet starter dog en postgis/postgis:16-3.4-service, så migrationen kan testes dér.

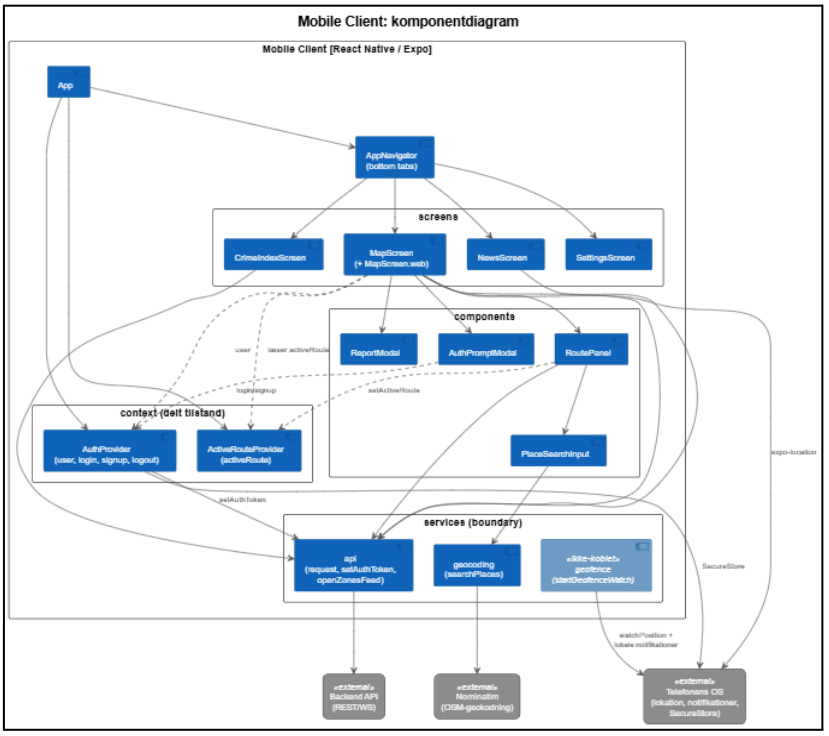
5.4.5 Delkonklusion

5.5 Delsystem Mobile Client

5.5.1 Introduktion

Mobile Client er den del af systemet, brugeren interagerer med. Den har ansvaret for kortvisning, rutesøgning og -valg (UC1), indrapportering (UC2), nyhedsfeed, bytryghedsoversigt og indstillinger, samt den lokale geofence-evaluering, som arkitekturen bevidst placerede på telefonen (afsnit 4.3). Den er skrevet i JavaScript med React Native og Expo og kører på iOS og Android fra samme kodebase (NF12), samt i en begrænset web-udgave.

5.5.1.1 Struktur af delsystem



Figur 5.6 viser komponentstrukturen. App pakker hele appen ind i to React-contexts og en tab-navigator med fire faner (Map, News, Crime Index, Settings):

Contexts (delt tilstand): AuthProvider holder den indloggede bruger og eksponerer login, signup og logout. ActiveRouteProvider holder den valgte rute. En context er valgt frem for at sende data via props, fordi rutevalget sker i RoutePanel, mens tegningen sker i MapScreen, og fordi der findes to varianter af kortskærmen (se nedenfor).

Screens: én pr. fane. MapScreen.js bruger react-native-maps; MapScreen.web.js vælges automatisk af Metro på web, hvor react-native-maps ikke findes, og viser i stedet en liste.

Components: RoutePanel (fra/til-søgning og ruteliste, foldet sammen over kortet), PlaceSearchInput, ReportModal og AuthPromptModal.

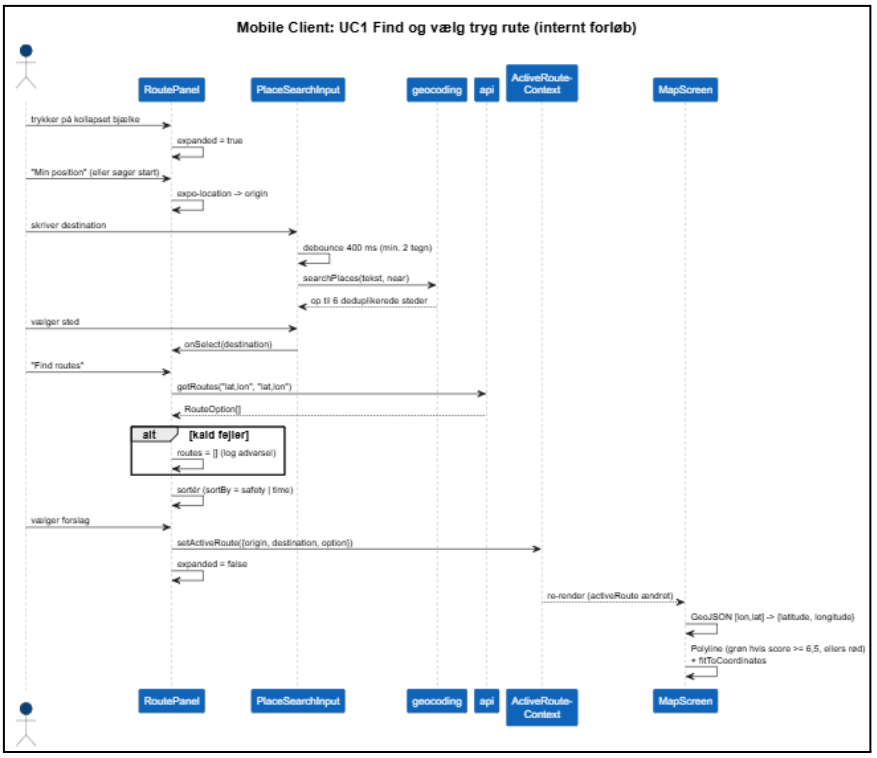
Services (boundary mod omverdenen): api (Backend API), geocoding (Nominatim) og geofence (telefonens lokation og notifikationer).

Ingen skærm eller komponent kalder fetch direkte; alle kald til Backend API går gennem services/api.js. Det samler tre ansvar ét sted: basis-URL'en (EXPO_PUBLIC_API_BASE_URL), automatisk Authorization: Bearer-header (sat af AuthProvider via setAuthToken, så kaldstederne ikke kender til tokenet) og fejlhåndtering, hvor FastAPI's detail-felt omsættes til en læsbar besked. Ved valideringsfejl (422) er detail en liste, som samles til én tekst.

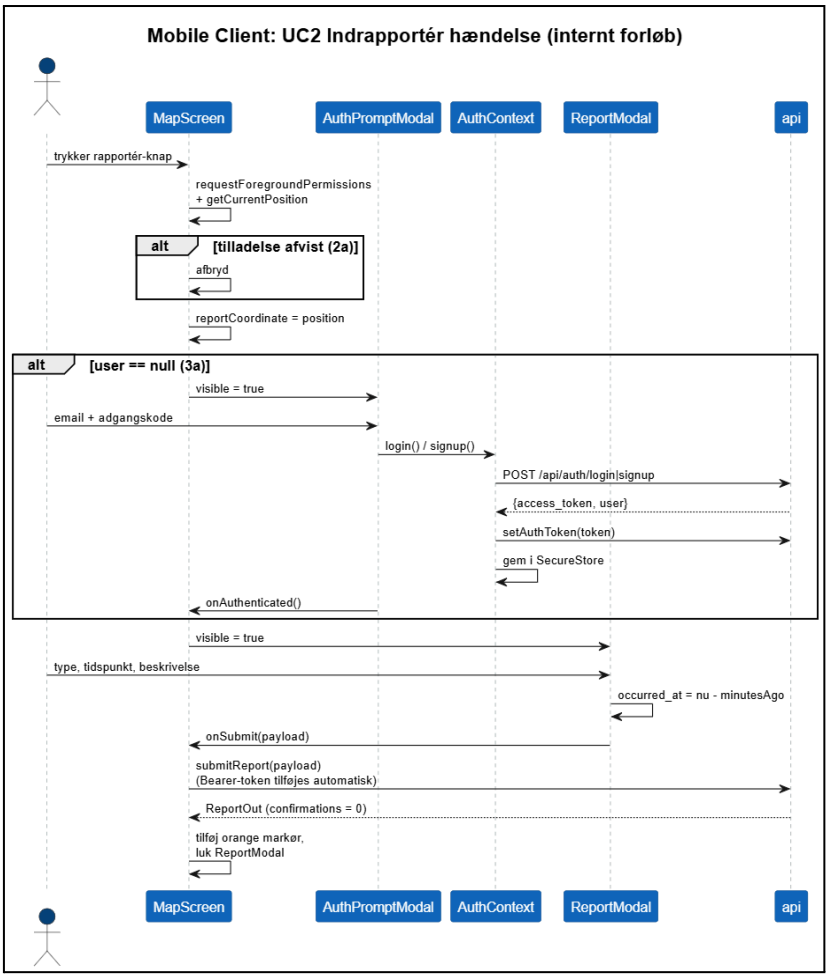
5.5.1.2 Interaktion mellem komponenter

Figur 5.7 viser UC1 internt i klienten. Stedsøgningen i PlaceSearchInput venter 400 ms efter sidste tastetryk og kræver mindst to tegn, før geocoding.searchPlaces kaldes. Det holder klienten under Nominatims grænse på ca. én forespørgsel i sekundet. Resultaterne vægtes mod brugerens position (blødt viewbox på ±0,2°) og dedupliceres på gade + by. Når en rute vælges, skrives den til ActiveRouteContext, og MapScreen reagerer

ved at tegne GeoJSON-geometrien som en polyline (grøn ved score ≥ 6,5, ellers rød) og zoome kameraet ind på den.



Figur 5.8 viser UC2. Positionen hentes, før login kontrolleres, så rapportens koordinat er brugerens aktuelle position (UC2 trin 2). Er brugeren ikke logget ind, åbnes AuthPromptModal først, og rapportformularen åbnes kun via onAuthenticated. Afbryder brugeren login, oprettes der ingen rapport (udvidelse 3a). Login er derimod ikke krævet for at søge en rute (NF9).



5.5.2 Design og implementering af kortskærmen (MapScreen)

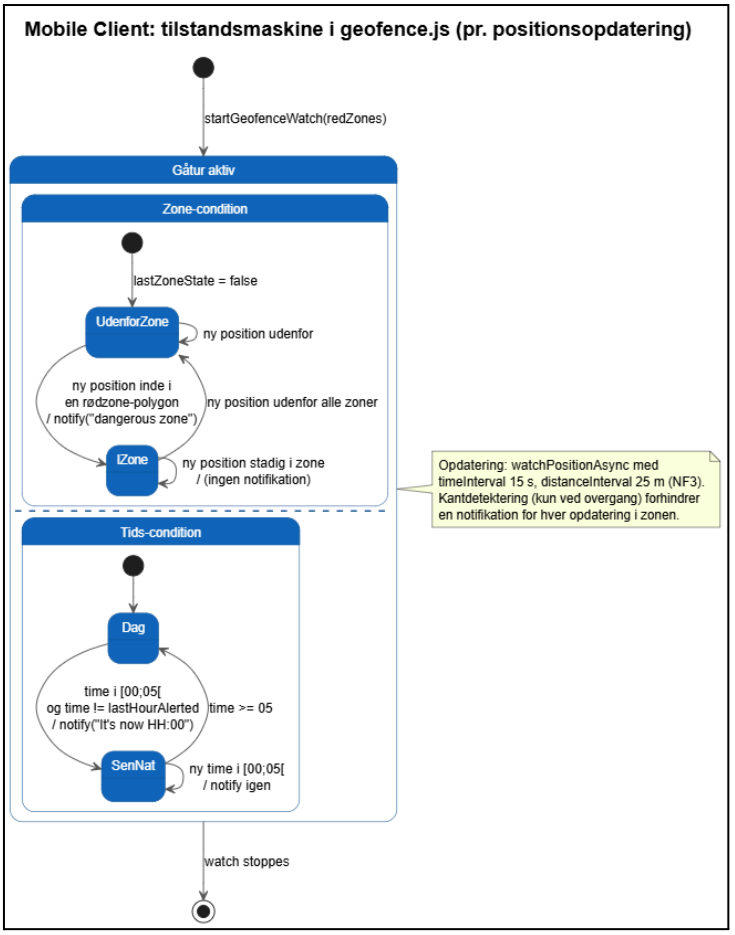
MapScreen viser tre slags markører: hændelser (rød, fra /api/incidents, p.t. tom), community-rapporter (orange, fra /api/reports) og kriminalitetsnyheder (blå). Rapporter og hændelser hentes igen, når kortets centrum flytter sig, for den synlige bbox. Nyheder er landsdækkende og hentes derfor kun én gang, for alle arkiverede år.

Nyheder placeres i et bycentrum (afsnit 5.3.3), så mange historier om samme by ville ligge på præcis samme punkt, og kun den øverste kunne trykkes på. jitterCoordinate spreder derfor hver markør deterministisk 450 m–1,1 km ud fra centret, med en hash af URL'en som frø, så den samme nyhed altid lander samme sted. Et tryk på en markør viser et informationskort med type, tidspunkt og beskrivelse (F8).

Farvekodede zoner (F7) vises ikke i den nuværende version; api.getZones er implementeret, men kaldes ikke fra kortskærmen.

5.5.3 Design og implementering af geofencing (services/geofence.js)

Geofence-modulet realiserer den klientlokale løsning fra afsnit 4.3 (figur 4.5). startGeofenceWatch(redZones) starter Location.watchPositionAsync med timeInterval: 15000 og distanceInterval: 25, svarende direkte til NF3, og evaluerer to uafhængige conditions for hver ny position. Logikken er vist som tilstandsmaskine i figur 5.9.



Zone-condition: et point-in-polygon-tjek (ray casting, uden eksterne biblioteker) mod hver rødzone. Der notificeres kun ved overgangen fra udenfor til indenfor (lastZoneState), så brugeren ikke får en ny notifikation hvert 15. sekund, mens vedkommende er i zonen.

Tids-condition: mellem kl. 00 og 05 notificeres der én gang pr. ny time (lastHourAlerted).

Notifikationen planlægges lokalt med scheduleNotificationAsync(..., trigger: null), dvs. med det samme og uden et kald til Backend API.

Modulet er implementeret, men ikke koblet på appen endnu: startGeofenceWatch kaldes ingen steder, og indstillingerne for zone- og tidsadvarsler i SettingsScreen er kun lokal skærmtilstand, der ikke gemmes eller læses af modulet. Desuden forventer modulet hver zone som {polygon: [{latitude, longitude}, ...]}, mens /api/zones returnerer GeoJSON ([lon, lat]-ringe, evt. MultiPolygon), så der mangler en lille adapter mellem de to. Point-in-polygon-funktionen er heller ikke eksporteret og derfor ikke enhedstestet.

5.5.4 Tests og resultater

Mobile Client har ingen automatiserede enheds- eller komponenttests; CI kører kun ESLint (npm run lint). Klienten er testet manuelt end-to-end mod en lokalt kørende Backend API via Expo. Backend-loggen fra en sådan session viser bl.a. POST /api/reports → 201, GET /api/routes → 200, GET /api/reports og GET /api/incidents → 200 (22 gange hver, ved kortbevægelser), GET /api/crime-index → 200 og GET /api/news → 200. UC1 og UC2's hovedforløb er dermed gennemført fra appen mod den rigtige backend.

Næste skridt er at tilføje Jest med jest-expo, så i hvert fald isInsidePolygon, fejlbeskedhåndteringen i api.request og dedupliceringen i geocoding kan testes isoleret.

5.5.5 Delkonklusion

Mobile Client er opbygget med en klar adskillelse mellem skærme, genbrugelige komponenter, delt tilstand i contexts og services, der indkapsler al kommunikation med Backend API, Nominatim og telefonens OS. UC1 og UC2 er implementeret fuldt ud og verificeret manuelt mod den rigtige backend, og NF9 (rute uden login) er overholdt. Den klientlokale geofencing, som er arkitekturens centrale valg for F5 og NF3, er implementeret som modul, men skal kobles på kortskærmen sammen med zonevisningen (F7) og en GeoJSON-adapter, før F5 er opfyldt.

6. Integrationstest

Kildeliste

1. Justitsministeriets Forskningskontor, Tryghedsundersøgelse 2023
2. Det Kriminalpræventive Råd (via Kristeligt Dagblad, 2021)
3. DR (2021), "Kvinder har nøgler og hårspray klar..."
4. Stiften (2022), "Johanne går ofte alene hjem om natten..."
5. Københavns Kommune, Tryghedsundersøgelsen 2025
6. Gallup, Global Law and Order
7. UK Office for National Statistics, tryghedstal for kvinder
8. "What do pedestrians consider when choosing a route?" (Monash/Cities, 2023)
9. "Coolight: Enhancing Nighttime Safety for Urban Student Commuters" (arXiv, 2025)
10. "Gender differences in pedestrian safety perceptions" (ScienceDirect, 2025)
11. ZoneScout, SafePath, Safest Way (apps/projekter)
12. FastAPI-dokumentation, "Concurrency and async / await". https://fastapi.tiangolo.com/async/
13. FastAPI-dokumentation, "WebSockets". https://fastapi.tiangolo.com/advanced/websockets/
14. Python Software Foundation, Python Glossary, opslag "global interpreter lock". https://docs.python.org/3/glossary.html#term-global-interpreter-lock
15. Django-dokumentation, "Asynchronous support". https://docs.djangoproject.com/en/stable/topics/async/
16. Flask-dokumentation, "Using async and await". https://flask.palletsprojects.com/en/stable/async-await/
17. Expo-dokumentation, "Location" (expo-location). https://docs.expo.dev/versions/latest/sdk/location/
18. Expo-dokumentation, "Notifications" (expo-notifications). https://docs.expo.dev/versions/latest/sdk/notifications/
19. W3C, Geolocation (W3C Recommendation), afsnit 6.5 "Request a position" og 6.6 "Acquire a position". https://www.w3.org/TR/geolocation/
20. WebKit, "Web Push for Web Apps on iOS and iPadOS" (2023). https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/
21. PostGIS-dokumentation, kap. 4 "Data Management" (afsnit om Geography-typen og spatiale indeks). https://postgis.net/docs/using_postgis_dbmanagement.html
22. MongoDB-dokumentation, "Geospatial Queries". https://www.mongodb.com/docs/manual/geospatial-queries/
23. MySQL 8.4 Reference Manual, "Spatial Analysis Functions". https://dev.mysql.com/doc/refman/8.4/en/spatial-analysis-functions.html
24. SQLite, "Appropriate Uses For SQLite". https://www.sqlite.org/whentouse.html
25. Luxen, D. & Vetter, C. (2011). "Real-time routing with OpenStreetMap data". Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, s. 513–516.
26. Geisberger, R., Sanders, P., Schultes, D. & Delling, D. (2008). "Contraction Hierarchies: Faster and Simpler Hierarchical Routing in Road Networks". Experimental Algorithms (WEA 2008), LNCS 5038, s. 319–333. Springer.
27. Project OSRM, "OSRM API Documentation" (v5.24.0), Route service. https://project-osrm.org/docs/v5.24.0/api/
28. GraphHopper, "Custom Models". https://github.com/graphhopper/graphhopper/blob/master/docs/core/custom-models.md
29. Valhalla-dokumentation. https://valhalla.github.io/valhalla/
30. Google Maps Platform, "Pricing". https://mapsplatform.google.com/pricing/
31. Celery-dokumentation, "Periodic Tasks". https://docs.celeryq.dev/en/stable/userguide/periodic-tasks.html